

DOSSIER DE PROJET • TITRE PROFESSIONNEL DWWM

RUNES ET DESTIN

Un voyage entre runes, rêve et spiritualité



Céline Devaux • La Plateforme, Cannes • Session 2026

Plan du dossier

Table des matières — dossier complet

Table des matières — dossier complet	2
Un projet qui réunit mon parcours et mes passions	3
Les compétences front-end travaillées	4
Installer et configurer mon environnement de travail	4
Maquetter les interfaces utilisateur	4
Réaliser des interfaces utilisateur statiques	4
Développer la partie dynamique des interfaces	4
Ce que je veux montrer au jury	4
Vision, public visé et besoins	5
Le site que j'aurais aimé trouver	5
À qui s'adresse l'application ?	5
Trois parcours différents	5
Cahier des charges et évolutions	6
Fonctions retenues pour cette première version	6
Évolutions prévues	6
Recherches sur les sites consacrés aux runes	7
Ce que j'ai observé	7
Ce que j'en ai retenu pour mon projet	7
Inspirations et construction de l'univers visuel	8
De l'image à l'interface	8
Le tirage comme moment visuel	9
Création du logo et charte graphique	10
Un logo créé à partir de runes liées	10
Palette, typographies et éléments communs	10
Versions exactes et choix techniques du front-end	12
Pourquoi React ?	12
Pourquoi Vite et Tailwind CSS ?	12
GitHub, dépôt front et historique des commits	13
Ce que montrent mes commits	13
GitHub Projects, tickets et sprints	15
Une organisation adaptée à un projet individuel	15
Arborescence et rangement des fichiers	16
Le rôle des dossiers principaux	16
Wireframes : organiser avant de décorer	17
Accueil	17

Connexion.....	18
Administration.....	18
Maquettes desktop et mobile	18
De la maquette au code	19
Layouts et routes : construire les trois espaces	20
GuestRoute et ProtectedRoute	21
ProtectedRoute	22
GuestRoute.....	22
Pages et composants réutilisables	22
Une page correspond à un écran	22
Un composant est un élément réutilisable	23
Responsabilité de chaque élément.....	23
Navbar et navigation responsive	23
Pourquoi séparer NavWeb et NavMobile ?	24
HomeCard : créer une carte réutilisable	25
Les props	25
Pourquoi cette page représente une interface statique	25
États React et déroulement d'un tirage	26
Lecture du code étape par étape	26
Les autres parties dynamiques.....	26
Faire vivre le tirage sans perdre la logique.....	28
Les états utilisés	28
Éviter les doubles clics et guider la lecture	28
Le Cercle d'Odin	28
Connexion : séparer la page et le formulaire	30
Connexion : validation, redirection et service.....	31
Le hook UseLogin	31
Le service AuthService.....	31
Dashboard membre et historique des tirages.....	32
Supprimer un résultat sans recharger toute la page	32
Télécharger et présenter les résultats	33
Dashboard administrateur et statistiques.....	33
Chargement des statistiques.....	34
Les autres blocs	34
Gestion des runes : créer, modifier et actualiser	34
Un seul formulaire pour deux actions.....	35
Responsive design avec Tailwind CSS	36
Adaptations principales.....	36
Vérifier le résultat sur plusieurs largeurs	36

Audit Lighthouse : avant et après les corrections	37
Comment j'interprète ces résultats	38
Rendre la navigation plus compréhensible et utilisable au clavier	38
Décrire le bouton du menu mobile	38
Indiquer la page active	38
Aller directement au contenu	39
Améliorer les boutons	39
Alléger le chargement sans supprimer l'univers visuel	39
Charger les pages seulement lorsqu'elles sont demandées	39
Choisir une image adaptée à l'écran	40
Donner la priorité au visuel principal	40
Optimiser les polices et les médias	40
Helmet Async, sitemap et robots.txt	41
Mettre en place HelmetProvider	41
Le fichier sitemap.xml	41
Modèle de données : le MCD	42
Du MLD au modèle physique	44
Dictionnaire des données — cœur fonctionnel	45
Pourquoi séparer rôles et catégories ?	45
Éviter les répétitions	45
Dictionnaire et schéma relationnel	46
Relations essentielles	46
Intégrité attendue	46
Sécurité de la base de données	46
API REST et architecture MVC	47
MVC : de la route au contrôleur	48
La route : le point d'entrée	48
Le contrôleur : validation et réponse HTTP	48
MVC : modèle, réponse JSON et CORS	49
Le modèle : accès aux données	49
Le CORS : autoriser le front sans ouvrir l'API à toutes les origines	49
Parcours complet d'une requête	49
JWT et gestion des rôles	50
Génération des PDF	51
Construction du document	51
Gestion des tirages et logique métier	51
Authentifier, protéger, valider et répondre proprement	52
Mesures complémentaires	52
Thunder Client, jeux d'essais, vérifications	53

De l'environnement local à la mise en ligne.....	54
Préparation de l'environnement.....	54
La configuration générale sur Hostinger est la suivante :.....	54
Mise en ligne	55
Faire grandir Runes et Destin sans repartir de zéro.....	56
Ce que ce projet m'a apporté.....	57
Retour vers le développement après dix ans d'aide à domicile	57
Mes objectifs futurs	57

Prologue

Un projet qui réunit mon parcours et mes passions

Dans le cadre de ma reconversion professionnelle, j'ai choisi de créer « Runes et Destin », le projet que je présente pour mon examen de Développeuse web et web mobile. Je m'appelle Céline Devaux et je suis élève à La Plateforme de Cannes. Ce site représente pour moi bien plus qu'un exercice technique : c'est un projet dans lequel j'ai mis une part de mon histoire, de mes compétences et d'un univers qui m'accompagne depuis longtemps.

Mon parcours a commencé par des études de graphisme, puis par plusieurs années de travail en freelance. J'ai collaboré avec des agences de communication, comme Youness Design et Looping Communication, mais aussi avec des clients directs : Le Bon Poulet, Salons Voyance Cannes, Biosphère Cannes, Biocoop, Laurie Lumière, entre autres. Ces expériences m'ont appris à écouter, à comprendre une demande, à créer une identité visuelle et à expliquer mes choix avec simplicité, même à des personnes qui ne connaissent pas forcément les outils utilisés.

La vie m'a ensuite amenée à mettre cette activité entre parenthèses. Pendant dix ans, j'ai travaillé comme aide à domicile. Cette expérience, profondément humaine, m'a beaucoup apporté : de l'écoute, de la patience, de l'attention aux autres et un vrai sens des responsabilités. Même si mon chemin a pris une autre direction pendant un temps, je n'ai jamais perdu mon attirance pour l'informatique, le graphisme et le développement. Ma formation à La Plateforme m'a permis de revenir progressivement vers un métier qui me ressemble davantage.

J'ai toujours été attirée par l'informatique, l'art et la spiritualité. Réunir ces trois domaines dans un même projet m'a donc semblé naturel, presque évident. Les runes, à la fois écriture ancienne et support symbolique, m'offraient un univers riche, mystérieux et inspirant à explorer. Elles sont liées aux peuples germaniques et sont encore aujourd'hui utilisées dans différentes pratiques, comme la divination, les talismans ou les soins énergétiques.

Pour cette première version, j'ai choisi de me concentrer sur la divination. Le site propose deux tirages : le Tirage des Nornes, avec trois runes, et le Cercle d'Odin, avec neuf runes. L'utilisateur peut choisir un domaine, découvrir les interprétations, retrouver ses tirages dans son espace membre et télécharger un PDF. J'ai aussi ajouté un glossaire pour accompagner les personnes qui découvrent les runes et leur permettre d'entrer plus facilement dans cet univers.

Je suis membre fondatrice de l'association EVA depuis 2001 et j'anime aujourd'hui une communauté Facebook de plus de 14 000 membres. Ce projet ne vient donc pas d'un thème choisi au hasard pour l'examen. Il s'appuie sur une passion sincère, présente dans ma vie depuis plus de trente ans, et sur l'envie de créer un outil utile, accueillant et vivant, que je pourrai continuer à faire évoluer après ma formation.

Mon objectif : Créer un site qui me ressemble, partager ce que j'ai appris et poser une base suffisamment solide pour continuer à faire grandir ce projet après l'examen, avec la même envie et le même engagement.

Référentiel DWWM

Les compétences front-end travaillées

Le projet m'a permis de travailler les quatre compétences attendues pour la partie front-end. Je les présente ici avec des mots simples, puis je détaille dans les pages suivantes ce que j'ai réellement réalisé.

Installer et configurer mon environnement de travail

J'ai travaillé sous Linux à La Plateforme et sous Windows chez moi. J'ai installé Node.js, npm et les dépendances du projet. J'ai aussi utilisé Git et GitHub pour retrouver le même code sur mes deux ordinateurs et conserver l'historique de mon travail.

Maquetter les interfaces utilisateur

Avant de coder, j'ai étudié plusieurs sites sur les runes, défini une ambiance, dessiné des wireframes puis réalisé les maquettes dans Figma. J'ai prévu dès cette étape la version ordinateur et la version mobile, ainsi que les écrans publics, membre et administrateur.

Réaliser des interfaces utilisateur statiques

J'ai d'abord construit la partie visible avec du contenu fixe : la page d'accueil, les menus, les cartes, les formulaires et les tableaux de bord. Cela m'a permis de vérifier l'organisation des écrans et le rendu de la charte avant d'ajouter les données réelles.

Développer la partie dynamique des interfaces

J'ai ensuite relié les écrans aux données. Les formulaires contrôlent les saisies, les tirages se déroulent étape par étape, le glossaire charge les runes, les tableaux de bord récupèrent leurs informations et l'affichage change selon la personne connectée.

Ce que je veux montrer au jury

Je ne souhaite pas seulement présenter le résultat. Je veux pouvoir expliquer le rôle de chaque partie : ce qui affiche l'écran, ce qui mémorise une valeur, ce qui appelle le serveur et ce qui met l'interface à jour.

Lecture du dossier : Les extraits présentés sont du code sélectionnable issu de mon projet. Je n'utilise pas de captures d'écran pour le code.

Fondation du projet

Vision, public visé et besoins

Le site que j'aurais aimé trouver

« Runes et Destin » est né d'un constat assez simple. En recherchant des informations et des tirages en ligne, j'ai trouvé beaucoup de sites anciens, peu immersifs ou limités à un générateur très basique. Je ne voulais pas faire un site de plus sur les runes : je voulais faire le site que j'aurais aimé trouver.

Mon idée était de proposer un univers complet, mais sans perdre l'utilisateur. La personne doit comprendre où elle se trouve, ce qu'elle peut faire et ce qu'elle obtiendra. Le design doit créer une ambiance, tandis que la navigation et les textes doivent l'accompagner.

À qui s'adresse l'application ?

Le premier public est constitué des personnes qui découvrent les runes. Elles ont besoin d'explications claires, d'un glossaire et d'un tirage simple pour commencer. Le second public connaît déjà les runes et recherche des interprétations plus complètes, un historique et la possibilité de conserver ses résultats.

Les échanges dans ma communauté m'ont donné une vision concrète des questions posées et des difficultés rencontrées. Je n'ai pas réalisé d'étude de marché formelle, mais je me suis appuyée sur cette expérience pour définir les besoins prioritaires.

Trois parcours différents

Profil	Ce qu'il peut faire	Besoin principal
Visiteur	Découvrir le site et consulter le glossaire	Comprendre l'univers
Membre	Réaliser, sauvegarder et télécharger ses tirages	Retrouver ses résultats
Administratrice	Gérer les runes, les membres et les tirages	Faire vivre le contenu

L'administration faisait partie des besoins dès le départ. Je voulais pouvoir ajouter ou corriger une rune depuis une interface, sans devoir ouvrir directement les fichiers ou modifier les données à la main.

Direction choisie : Une application assez simple pour une personne débutante, mais assez complète pour accompagner une pratique régulière.

Fondation du projet

Cahier des charges et évolutions

J'avais beaucoup plus d'idées que de temps pour les réaliser. J'ai donc défini une première version utilisable, souvent appelée MVP. Cela ne veut pas dire une version vide ou bâclée : il s'agit d'une base qui couvre le parcours principal et qui pourra ensuite être enrichie.

Fonctions retenues pour cette première version

Fonction	Réponse au besoin	État
Accueil immersif	Présenter immédiatement l'univers du site	Réalisé
Inscription et connexion	Créer et retrouver son espace	Réalisé
Tirage des Nornes	Effectuer un tirage gratuit à 3 runes	Réalisé
Cercle d'Odin	Obtenir une lecture complète à 9 runes	Réalisé
Glossaire	Consulter les 24 runes et leurs sens	Réalisé
Historique membre	Retrouver ou supprimer ses tirages	Réalisé
Téléchargement PDF	Conservé et imprimer un résultat	Réalisé
Administration	Gérer le contenu depuis une interface	Réalisé

Évolutions prévues

- Ajouter le Marteau de Thor à cinq runes et la Tête de Mimir à sept runes.
- Créer des talismans runiques selon le thème choisi et les intentions de la personne.
- Enrichir le glossaire avec les sons, les positions du corps et les soins liés aux runes.
- Ajouter une partie historique consacrée aux runes et à la mythologie nordique.
- Améliorer la présentation du PDF et mettre en place la vente des tirages complets.

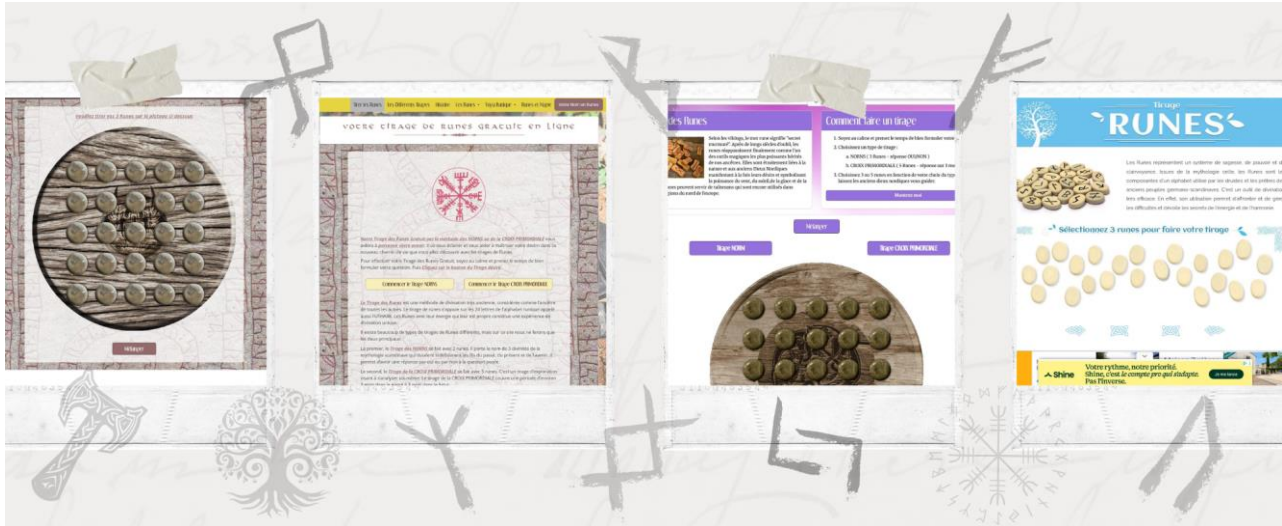
Le tirage à trois runes doit rester gratuit. Les autres formats pourront devenir payants, avec une tarification liée au nombre de runes. J'ai reporté cette partie afin de terminer les parcours essentiels et l'administration avant d'ajouter une logique commerciale.

Choix de priorité : J'ai préféré terminer les fonctions principales, les rendre compréhensibles et les relier correctement plutôt que commencer toutes mes idées sans pouvoir les finir.

Direction artistique

Recherches sur les sites consacrés aux runes

Avant de dessiner mes premiers écrans, j'ai recherché ce qui existait déjà. J'ai consulté différents sites de tirages et d'informations sur les runes. Cette étape m'a permis de voir leurs points communs, leurs qualités, mais aussi ce que je ne voulais pas reproduire.



Sélection de sites étudiés avant la création de ma propre direction graphique.

Ce que j'ai observé

- Beaucoup de pages utilisent un fond peu immersif, des symboles anciens et des couleurs dorées ou brunes.
- Les tirages sont souvent présentés comme un résultat immédiat, avec peu de mise en scène.
- Certains sites contiennent beaucoup de texte, mais la hiérarchie et la lecture sont parfois difficiles.
- Les versions mobiles et les espaces personnels sont rarement mis en valeur.

Ce que j'en ai retenu pour mon projet

Je voulais garder le mystère et les références nordiques, mais avec une interface plus moderne. J'ai donc conservé une palette nocturne, tout en ajoutant une vraie organisation des contenus, des boutons identifiables, des composants réutilisables et une expérience de tirage progressive.

Je n'ai pas cherché à copier une mise en page. Ces recherches m'ont surtout aidée à comprendre les codes visuels du sujet et à trouver la différence que je voulais apporter : un univers plus vivant, un accompagnement plus clair et une vraie continuité entre l'accueil, le tirage et l'espace membre.

Direction artistique

Inspirations et construction de l'univers visuel

Dès le début, j'ai imaginé une Völva au coin du feu, à la recherche des présages dans les runes. Cette scène est devenue le fil conducteur du projet. Elle apporte une présence humaine et crée une atmosphère avant même que l'utilisateur commence son tirage.



Je me suis servie de l'intelligence artificielle (chat GPT) pour explorer plusieurs pistes visuelles et préciser ce que je voulais. Je n'ai pas gardé la première proposition. J'ai comparé les ambiances, les cadrages, la place du personnage et la lumière jusqu'à obtenir une image qui pouvait accueillir le titre et les boutons.

De l'image à l'interface

Le visuel retenu a ensuite été animé pour la page d'accueil sur tablette et ordinateur. Sur téléphone, j'utilise une image fixe. La vidéo serait plus lourde et n'apporterait pas autant sur un écran étroit. Ce choix améliore le chargement tout en conservant la même ambiance.

J'ai également ajouté un voile bleu nuit au-dessus du média. Sans ce voile, certaines zones lumineuses rendaient le texte moins lisible. L'image reste visible, mais elle ne prend pas le dessus sur le contenu.

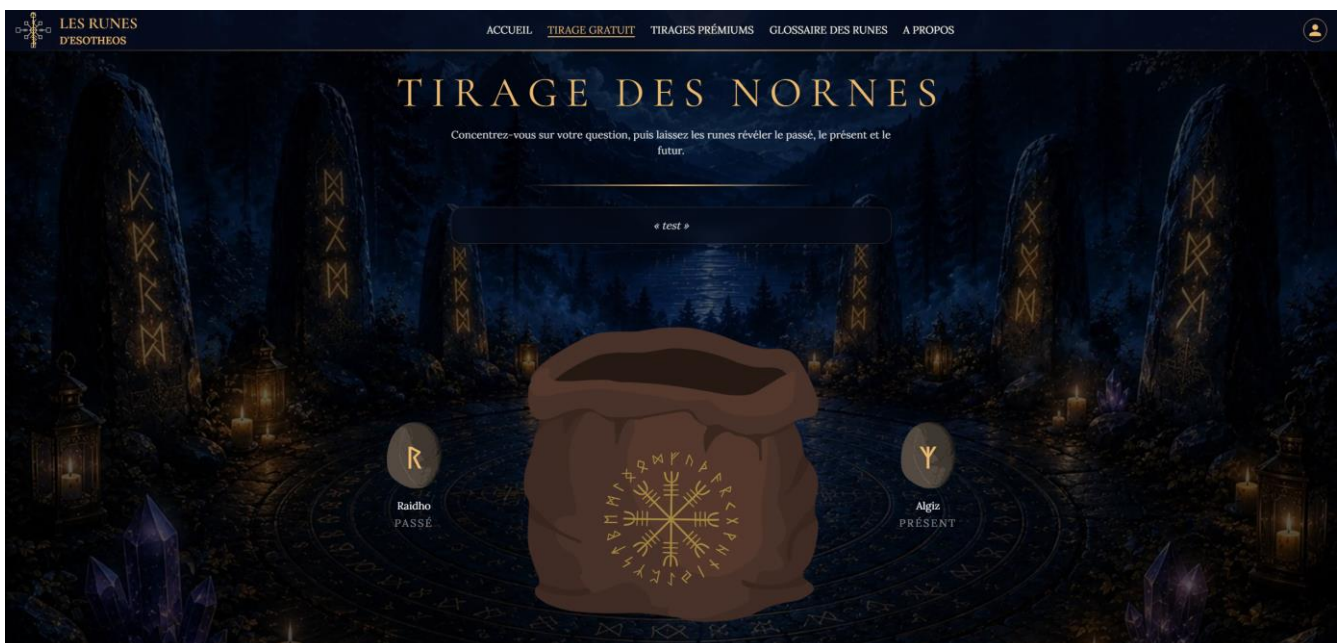
Afin d'explorer différentes pistes, je me suis appuyée sur l'intelligence artificielle pour générer plusieurs idées et affiner l'ambiance générale. Certains visuels m'ont permis de mieux définir l'atmosphère que je souhaitais retrouver dans l'application.



Ambiances retenues : forêt, feu, nuit, pierres, lumière dorée et symboles runiques.

Le tirage comme moment visuel

Je ne voulais pas afficher trois résultats dès le clic. J'ai préféré rappeler le geste du tirage traditionnel : le sac bouge, une pierre apparaît, se déplace puis révèle le symbole. Cette idée a guidé une grande partie des composants d'animation développés ensuite.



Mon intention : Créer une ambiance spirituelle et nordique, sans transformer le site en décor compliqué à utiliser.

Identité graphique

Création du logo et charte graphique

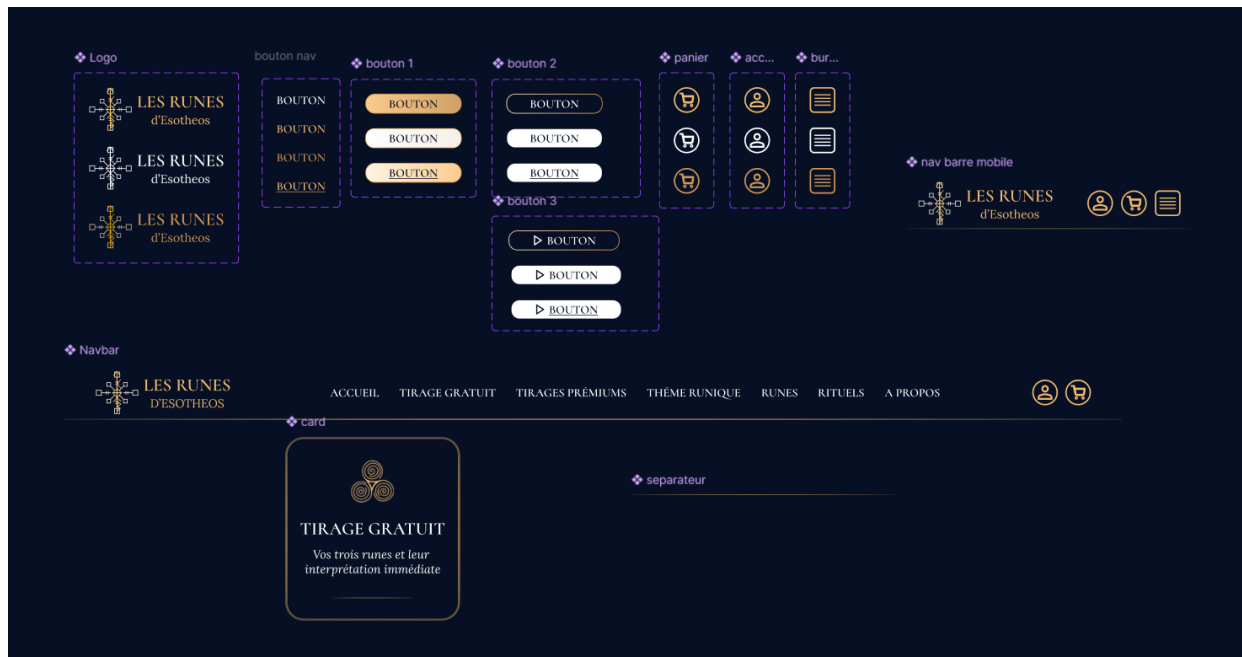
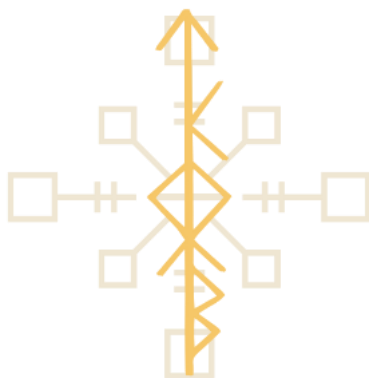


Planche de départ : logo, palette, navigation, boutons et principe des cartes.

Un logo créé à partir de runes liées

J'ai réalisé le logo moi-même en assemblant des formes inspirées des runes. Je voulais un symbole propre au projet, reconnaissable même sans le nom du site. La planche présente ses versions claire et dorée, ainsi que les premiers boutons et composants.



GREAT VIBE = TITRE H1

Je suis le titre de la page d'accueil

CORMORANT = TITRES

JE SUIS LES AUTRES TITRES

LORA = PARAGRAPHE

Je suis les paragraphes



BLEU CRÉPUSCULE

#070F24



TERRE DE SIENNE

#5C3E34



CHAMPAGNE DORÉ

#EFBB6F



VIEIL OR

#CF9752



LUEUR DE FEU

#FB9D30

Palette, typographies et éléments communs

Le bleu nuit #070F24 est le fond principal. Le doré #EFBB6F sert pour les titres, les bordures, les symboles et les actions importantes. Le blanc et les gris clairs sont réservés aux textes. J'utilise Cormorant Garamond pour les titres et Lora pour les contenus afin de garder une identité ancienne sans rendre la lecture difficile.

Les boutons, les cartes et les séparateurs suivent les mêmes règles sur tout le site. Cette cohérence graphique est ensuite traduite dans le code avec des composants React et des classes Tailwind. Si je corrige un composant, toutes les pages qui l'utilisent profitent de la modification.

Environnement

Versions exactes et choix techniques du front-end

J'ai développé le projet sous Linux à La Plateforme et sous Windows chez moi. Node.js et npm permettent d'installer les bibliothèques indiquées dans package.json. Le fichier package-lock.json conserve les versions réellement utilisées pour que l'installation soit identique sur une autre machine.

Outil ou bibliothèque	Version	Rôle dans mon projet
Node.js	24.13.1	Exécuter les outils du projet et npm
React / React DOM	19.2.6	Construire et afficher l'interface
React Router	7.15.1	Gérer les pages et les redirections
Tailwind CSS	4.3.0	Créer le style et le responsive
@tailwindcss/vite	4.3.0	Relier Tailwind à Vite
Vite	8.0.12	Lancer le serveur local et préparer le build
Plugin React pour Vite	6.0.1	Faire fonctionner React avec Vite
ESLint	10.3.0	Repérer les erreurs et incohérences du code

Pourquoi React ?

J'avais déjà utilisé React sur d'autres projets. Son découpage en composants correspondait bien à mon application, qui contient de nombreux éléments répétés : boutons, cartes, formulaires, lignes de tableaux, pierres et résultats de tirage. Je peux modifier un élément réutilisable sans reprendre chaque page.

Pourquoi Vite et Tailwind CSS ?

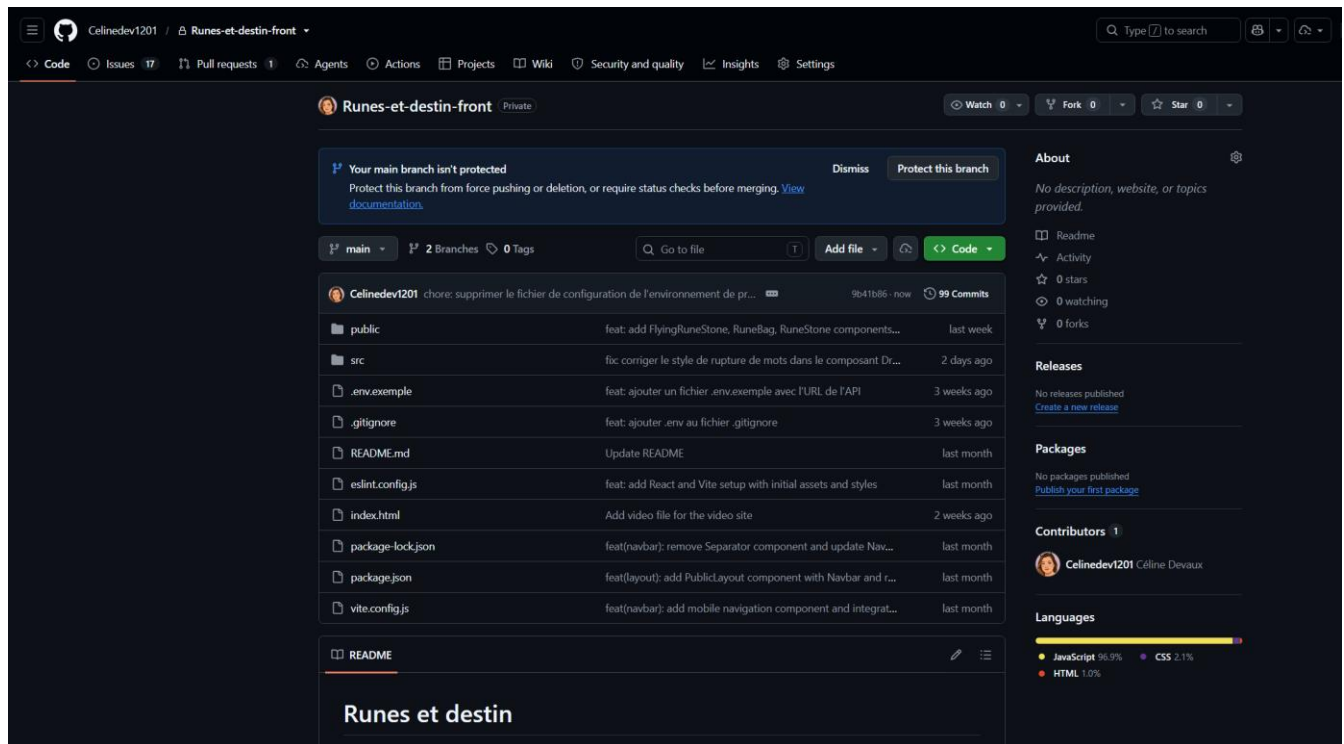
Vite me donne un démarrage rapide et affiche presque immédiatement mes changements pendant le développement. Tailwind CSS me permet de construire le style directement dans les composants et d'utiliser des paliers de largeur pour adapter les écrans. Je l'avais découvert sur le projet MarsAI et j'ai choisi de l'approfondir ici.

Outils de travail : J'ai utilisé Visual Studio Code pour développer, Figma pour les maquettes, Git et GitHub pour les versions et GitHub Projects pour organiser les tâches.

Ma manière de travailler

GitHub, dépôt front et historique des commits

Le front-end et le back-end sont rangés dans deux dépôts différents. Cette séparation me permet de faire évoluer l'interface sans mélanger les fichiers avec ceux de l'API. Dans cette partie du dossier, je présente uniquement le dépôt du front-end.



Github : le dépôt front

J'ai utilisé GitHub pour travailler sur mon ordinateur portable à La Plateforme puis reprendre chez moi. Je récupère les dernières modifications, je travaille sur une tâche précise, je vérifie le résultat puis j'enregistre une étape avec un commit.

Ce que montrent mes commits

Je n'ai pas attendu la fin du projet pour tout enregistrer en une seule fois. Les commits suivent le développement : animation des tirages, gestion du compte, historique membre, PDF, administration et corrections responsive. Les préfixes feat et fix m'aident à comprendre rapidement la nature d'une modification.

Commits on Jun 22, 2026		
fix: corriger le style de rupture de mots dans le composant DrawResultGridOdin	e0c9caa	 
 CelineDev1201 committed 2 days ago		
feat: ajouter la fonctionnalité de tirage de runes avec animation et interprétations	719e2a5	 
 CelineDev1201 committed 2 days ago		
feat: ajouter la fonctionnalité de suppression de compte avec confirmation par email	edc5adb	 
 CelineDev1201 committed 2 days ago		
Commits on Jun 18, 2026		
fix: corriger le style de rupture de mots dans les titres des composants RunesTable et DashboardMembre	f295165	 
 CelineDev1201 committed last week		
feat: ajouter la fonctionnalité de changement de mot de passe pour les membres	3ebaca2	 
 CelineDev1201 committed last week		
Commits on Jun 17, 2026		
feat: ajouter la gestion des runes tirées avec affichage et interprétations dans l'interface des membres	4306ffc	 
 CelineDev1201 committed last week		
feat: ajouter la fonctionnalité de téléchargement des résultats au format PDF dans l'interface admin	0b2686b	 
 CelineDev1201 committed last week		
Commits on Jun 16, 2026		
feat: ajouter la fonctionnalité de téléchargement des résultats des membres au format PDF	cc4ee5a	 
 CelineDev1201 committed last week		
feat: ajouter la gestion des tirages avec chargement, suppression et affichage des résultats dans l'interface admin	8cdb517	 
 CelineDev1201 committed last week		
fix: corriger la largeur minimale de la table des résultats des membres	97d5a20	 
 CelineDev1201 committed last week		
feat: ajouter la gestion des tirages avec navigation et boutons dans l'interface admin	fe04ca0	 
 CelineDev1201 committed last week		
feat: ajouter le tableau des résultats des membres avec gestion de la suppression	8fdbdb5	 
 CelineDev1201 committed last week		

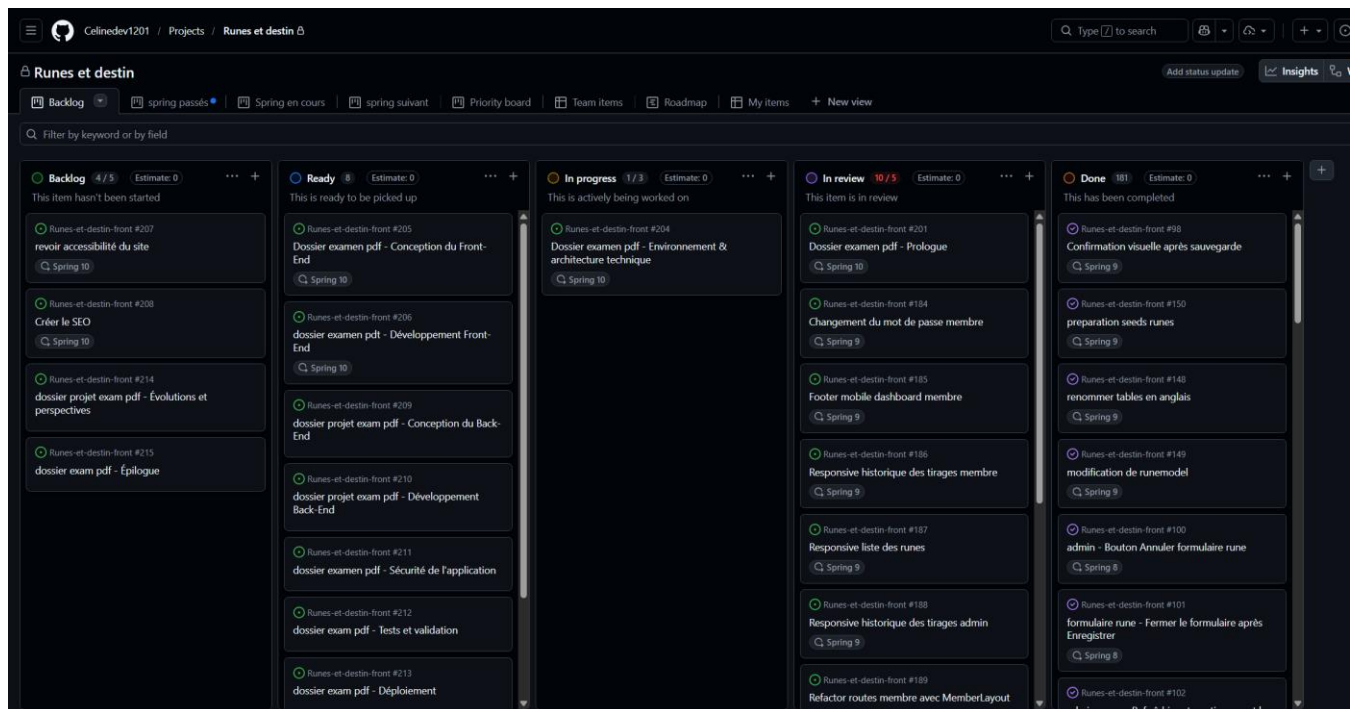
Github : une partie de l'historique des commits.

Cet historique me permet de revenir à une étape précédente, de retrouver l'origine d'un problème et de montrer comment le projet a avancé. Il fait donc partie de ma méthode de travail, au même titre que le code final.

Ma manière de travailler

GitHub Projects, tickets et sprints

Au fur et à mesure, mes idées devenaient nombreuses. Pour ne pas me perdre, j'ai utilisé GitHub Projects. J'y ai transformé les fonctionnalités, les corrections et les améliorations en tickets. Chaque ticket correspond à un travail identifiable, avec un objectif et un état d'avancement.



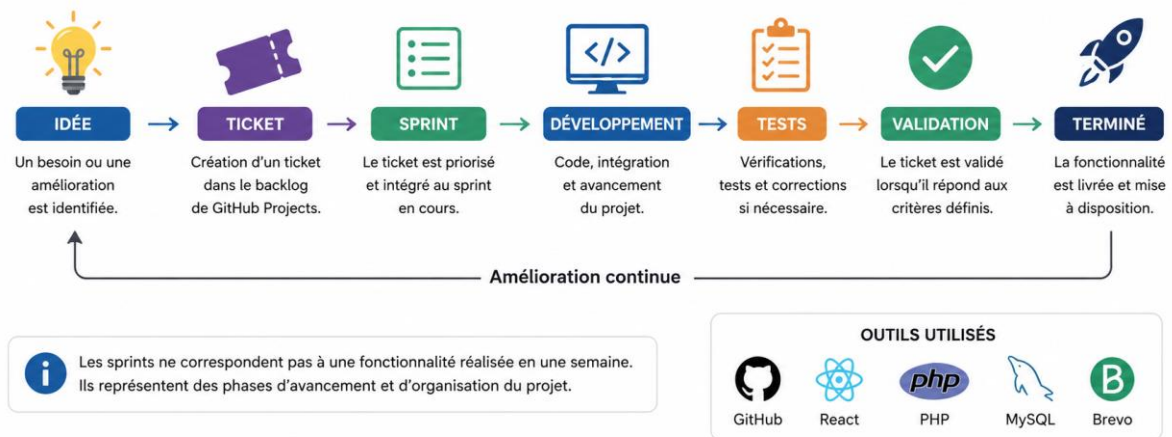
GitHub Projects et le déroulement suivi pour passer d'une idée à une tâche terminée.

Une organisation adaptée à un projet individuel

Je me suis inspirée des méthodes agiles, sans prétendre reproduire exactement le fonctionnement d'une grande équipe. J'ai organisé mon travail par sprints. Un sprint correspondait à une phase d'avancement : par exemple l'authentification, les tirages, l'espace membre, l'administration ou le responsive.

MÉTHODE AGILE

Organisation du projet avec GitHub Projects



Je commençais par choisir les tickets prioritaires. Une fois la fonction développée, je la vérifiais, je corrigeais les problèmes puis je déplaçais le ticket dans la colonne terminée. Les fonctions trop importantes pour le temps restant étaient replacées dans le backlog ou prévues pour une prochaine version.

Cette méthode m'a aidée à construire le site étape par étape et à garder une vue globale. Elle montre aussi que mon travail ne se résume pas à écrire du code : je dois choisir un ordre, suivre les changements et décider ce qui appartient réellement à la première version.

Conception du front-end

Arborescence et rangement des fichiers

Le projet a rapidement grandi. Pour retrouver facilement le code, j'ai classé les fichiers selon leur rôle. Ce rangement m'évite de mélanger les écrans, les éléments réutilisables et les appels vers le serveur.

Organisation simplifiée du dossier src

```
src/
├── components/  éléments réutilisables
│   ├── buttons/ boutons communs
│   ├── forms/   formulaires
│   ├── draws/   tirages et animations
│   └── tables/   listes membre et admin
├── pages/       écrans publics, membre et admin
├── layouts/     structures communes
├── routes/      contrôles d'accès
├── hooks/       logique réutilisable
└── services/    appels vers l'API
```

Le rôle des dossiers principaux

Dossier	Contenu
pages	Les écrans publics, membres et administrateur
components	Les éléments réutilisables : boutons, formulaires, tirages, tableaux...
layouts	La structure commune de chaque espace
routes et hooks	Les contrôles d'accès et la logique réutilisable
services	Les fonctions qui communiquent avec l'API
public	Les images, vidéos, icônes et ressources visibles

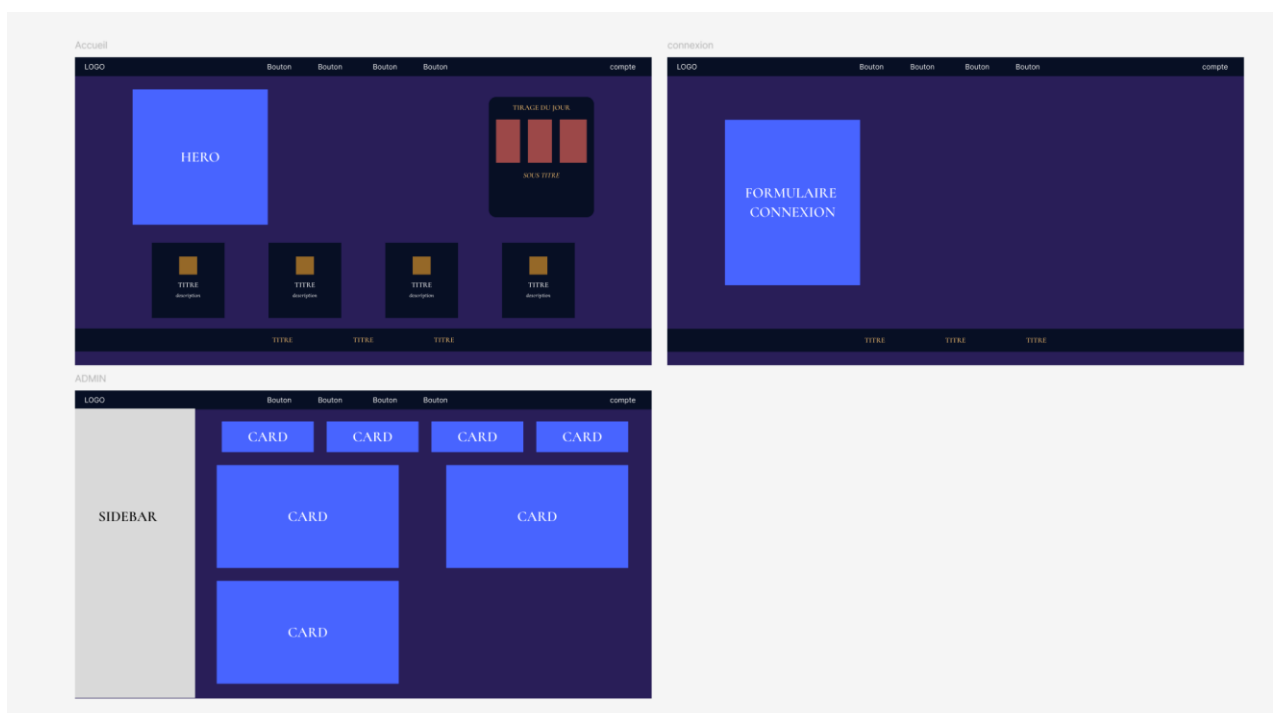
Par exemple, `DashboardMembre.jsx` se trouve dans `pages/membres`, alors que `MemberResultsTable.jsx` se trouve dans `components/tables`. La page décide quoi afficher ; le composant tableau s'occupe de la présentation de la liste.

Cette organisation n'a pas été parfaite dès le premier jour. Je l'ai améliorée au fur et à mesure que de nouvelles fonctions apparaissaient. Le résultat reste lisible et je sais aujourd'hui où intervenir pour modifier une page, un formulaire ou un appel de données.

Conception des interfaces

Wireframes : organiser avant de décorer

J'ai réalisé les wireframes au début du projet. Ils servent à placer les grands blocs sans se concentrer sur les couleurs ou les images. Cela m'a permis de réfléchir d'abord à la navigation et à l'ordre des informations.



Trois écrans de base : accueil public, connexion et tableau de bord administrateur.

Accueil

J'ai prévu une navigation principale, une grande zone d'introduction, des actions visibles et des cartes présentant les fonctions. Le but est que la personne comprenne rapidement l'univers et puisse choisir entre découvrir les runes ou commencer un tirage.

Connexion

Le formulaire est volontairement centré et isolé du reste. L'utilisateur doit identifier immédiatement les deux champs, l'action de connexion, le lien de mot de passe oublié et la possibilité de créer un compte.

Administration

Le tableau de bord utilise une barre latérale pour les outils et une zone centrale pour les données. J'avais besoin de distinguer clairement les statistiques, les membres, les tirages et la gestion des runes.

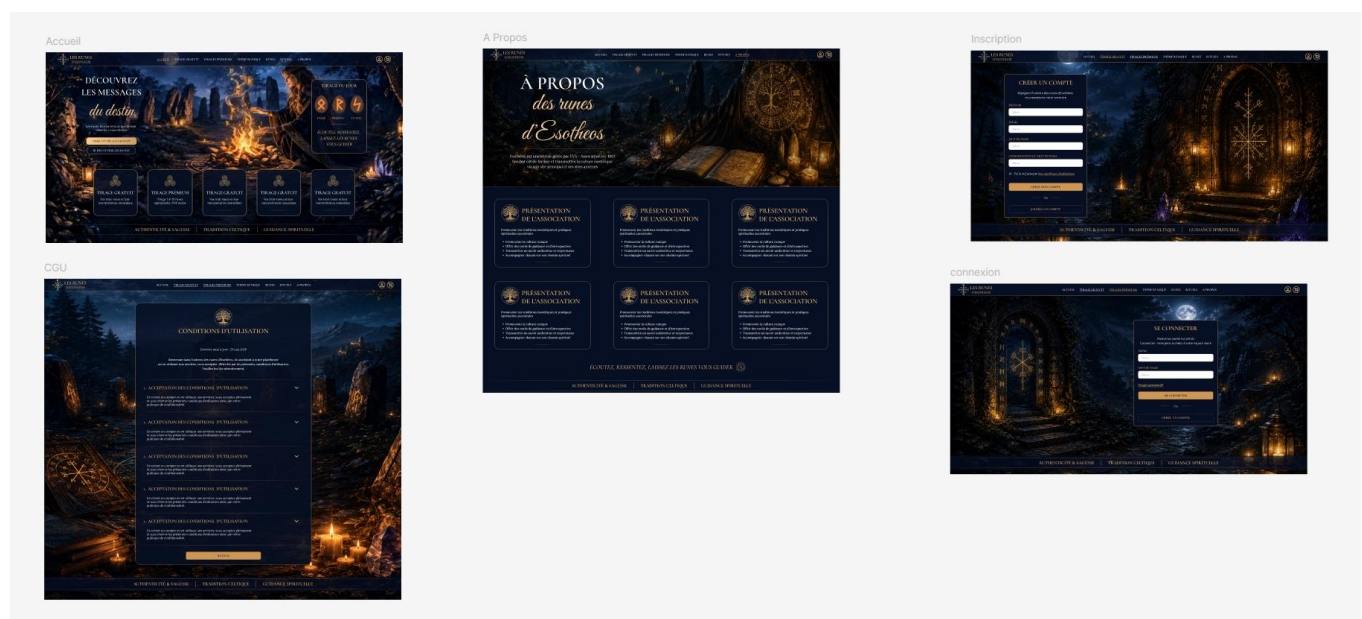
Les wireframes m'ont aussi permis d'anticiper les éléments réutilisables : mêmes boutons, mêmes cartes, mêmes zones de titre et mêmes principes de tableaux. J'ai ensuite repris cette structure dans les maquettes, avec la vraie identité graphique.

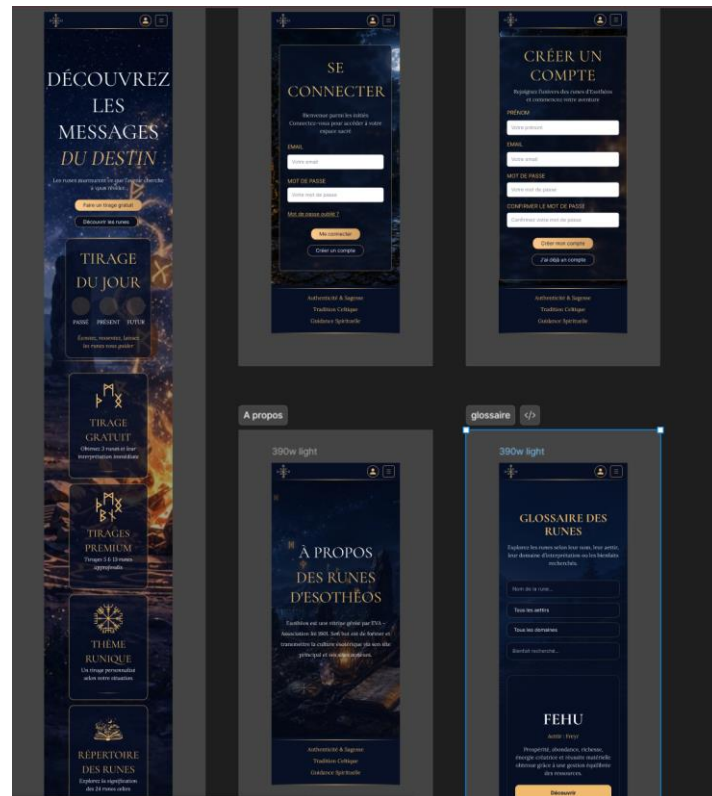
Ce que cette étape m'a évité : de commencer directement par le code et devoir déplacer toute l'interface parce que le parcours n'avait pas été réfléchi.

Conception des interfaces

Maquettes desktop et mobile

Une fois la structure validée, j'ai créé les maquettes dans Figma. J'ai ajouté la palette, les images, les typographies, les boutons, les espacements et les composants. Ces écrans ont servi de référence pendant le développement.



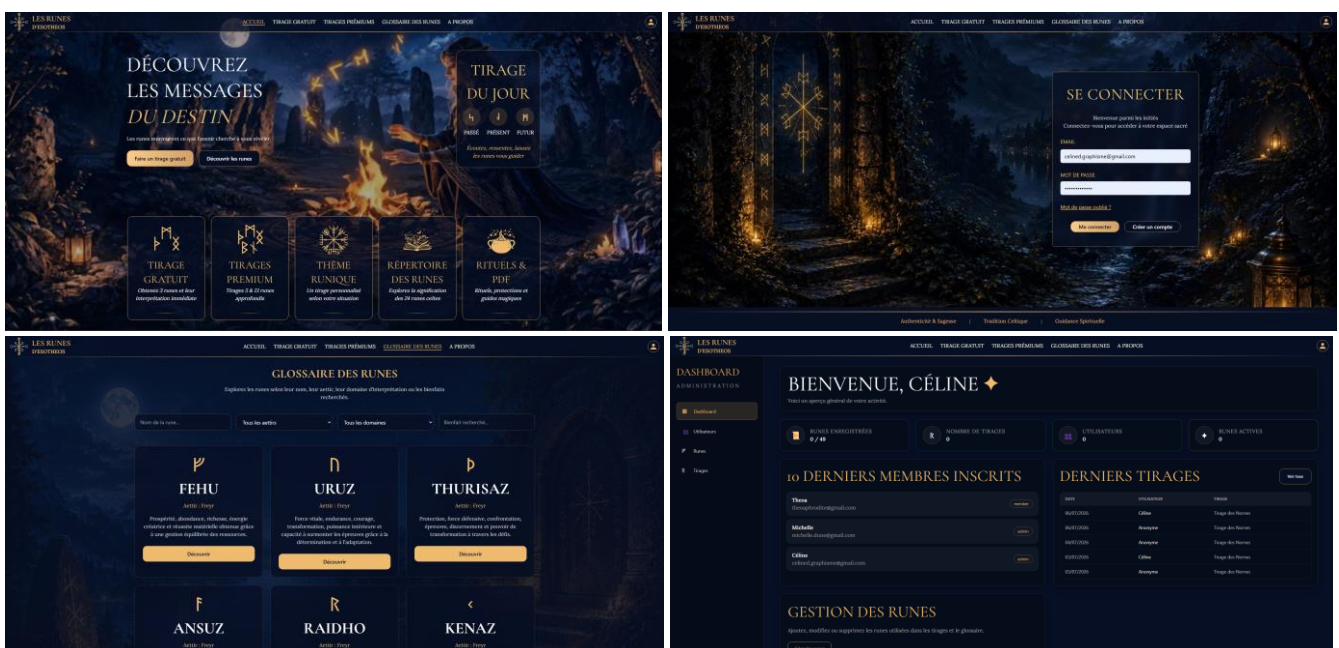


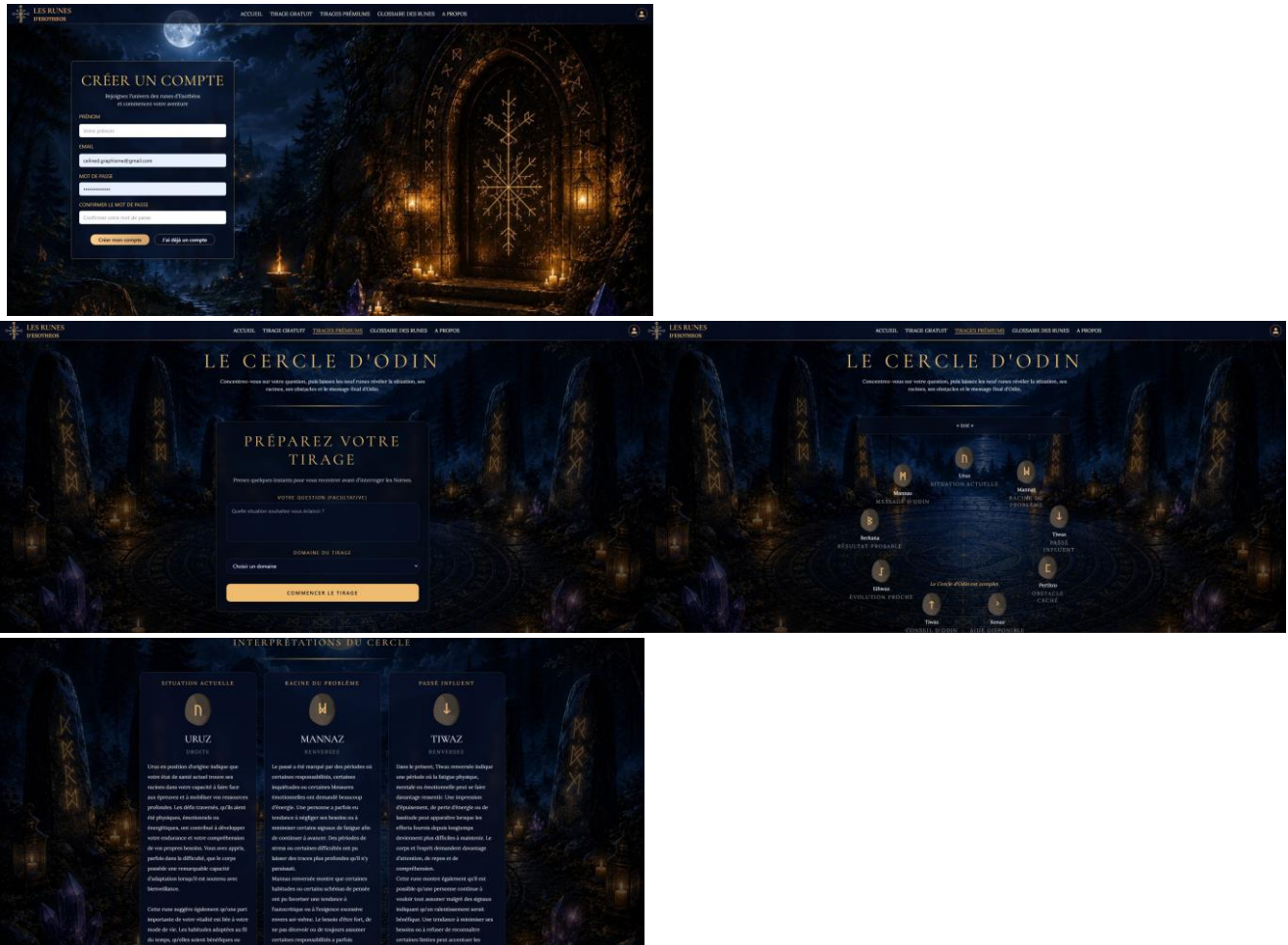
Les mêmes parcours ont été préparés en version ordinateur et mobile.

Sur ordinateur, j'ai laissé davantage de place aux visuels et utilisé plusieurs colonnes. Sur mobile, la navigation change, les cartes passent les unes sous les autres et les tableaux deviennent des fiches plus faciles à lire.

De la maquette au code

Le rendu final n'est pas une copie au pixel près. Certaines tailles et certains placements ont évolué pendant l'intégration, en fonction du vrai contenu et du comportement des données. La maquette reste un guide, tandis que le navigateur permet de vérifier la réalité de l'écran.

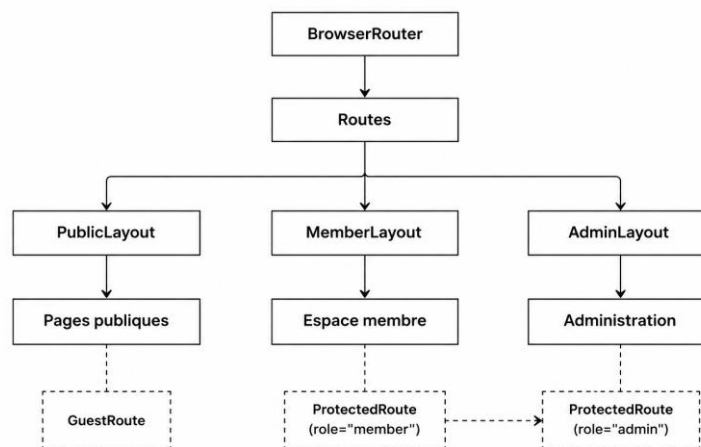




Structure React

Layouts et routes : construire les trois espaces

J'ai commencé par mettre en place trois layouts. Un layout est une structure commune qui entoure plusieurs pages. Cela évite de recopier la navigation, le pied de page ou la zone principale dans chaque fichier.



Les routes publiques, membres et administrateur utilisent chacune leur propre layout.

Layout	Pages concernées	Éléments partagés
PublicLayout	Accueil, glossaire, connexion, inscription	Navbar et pied de page
MemberLayout	Tirages, historique, compte	Navigation membre
AdminLayout	Dashboard et gestion	Sidebar et zone admin

Dans le fichier `src/layouts/PublicLayout.jsx`

```
export default function PublicLayout() {
  return (
    <div className="flex min-h-screen flex-col bg-[#070F24]">
      <Navbar />
      <main className="flex-1">
        <Outlet />
      </main>
      <HomeFooter />
    </div>
  );
}
```

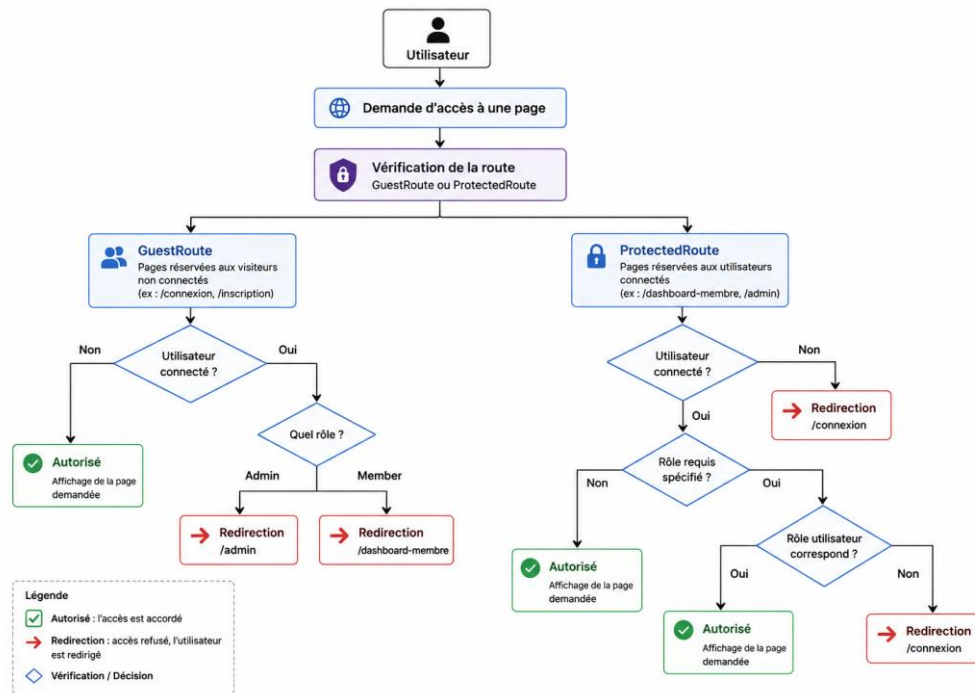
Outlet indique l'endroit où React Router doit afficher la page demandée. Si l'adresse correspond à l'accueil, l'accueil prend sa place. Si elle correspond au glossaire, le glossaire est affiché au même endroit. Navbar et HomeFooter restent communs.

Les routes sont centralisées dans `main.jsx`. Je peux ainsi voir dans un seul fichier les adresses disponibles, le layout utilisé et les éventuelles protections. La navigation se fait sans recharger complètement le navigateur, ce qui rend le passage d'une page à l'autre plus fluide.

Structure React

GuestRoute et ProtectedRoute

Les trois espaces ne sont pas ouverts de la même manière. Un visiteur peut voir les pages publiques. Un membre doit être connecté pour accéder à son historique. L'administration demande en plus le rôle admin. J'ai regroupé ces vérifications dans deux composants.



Selon la connexion et le rôle, la page est affichée ou l'utilisateur est redirigé.

ProtectedRoute

Dans le fichier `src/routes/ProtectedRoute.jsx`

```

if (!isLoggedIn()) {
  return <Navigate to="/connexion" replace />;
}

if (role && user?.role !== role) {
  return <Navigate to="/connexion" replace />;
}

return children;

```

Le composant vérifie d'abord si des informations de connexion existent. Si ce n'est pas le cas, `Navigate` renvoie vers la connexion. Lorsqu'un rôle est demandé, il compare ensuite ce rôle avec celui de l'utilisateur. La page n'est affichée qu'après ces vérifications.

GuestRoute

`GuestRoute` fait l'inverse pour les pages de connexion et d'inscription. Si une personne déjà connectée tente d'y revenir, elle est redirigée vers son propre espace : `/admin` pour l'administratrice ou `/dashboard-membre` pour un membre.

Centraliser cette logique m'évite de répéter les mêmes conditions dans chaque page. Si je dois changer la règle de redirection, je sais que la correction se trouve dans un seul composant.

Structure React

Pages et composants réutilisables

Une page correspond à un écran

Les pages sont classées dans pages/public, pages/membres et pages/admin. Une page assemble les éléments nécessaires à un écran et organise la logique générale. Par exemple, DashboardAdmin affiche l'en-tête puis appelle les composants des statistiques, des derniers membres, des derniers tirages et des runes.

Un composant est un élément réutilisable

Un composant React est un élément d'interface réutilisable, créé sous forme de fonction. Il peut représenter un bouton, une carte, un formulaire, un menu ou une partie plus complète d'un écran. Il reçoit éventuellement des informations, appelées props, puis retourne ce qui doit être affiché.

J'ai rangé mes composants par rôle afin de les retrouver facilement.

Dossier	Exemples	Utilité
buttons	ButtonGold, ButtonBlue	Réutiliser les mêmes actions
navbar	NavWeb, NavMobile, MenuNav	Adapter la navigation
forms	LoginForm, RuneForm	Afficher et contrôler les formulaires
draws	RuneBag, FlyingRuneStone	Construire les animations
glossaire	Cartes et filtres	Consulter les runes
tables	RunesTable, MemberResultsTable	Présenter les listes
deco	HomeCard, Separator	Partager les éléments graphiques

Responsabilité de chaque élément

J'essaie de donner un rôle principal à chaque composant. LoginForm affiche les champs et les messages. UseLogin gère la validation et la redirection. AuthService communique avec l'API. Cette séparation me permet de modifier l'apparence du formulaire sans réécrire la demande de connexion.

Tous les composants ne doivent pas être minuscules. Je crée un composant séparé lorsqu'un élément est réutilisé, lorsqu'il possède sa propre logique ou lorsque son extraction rend la page plus simple à comprendre.

Exemple concret : Un bouton doré utilisé sur dix pages ne doit pas être recopié dix fois. Le composant ButtonGold garde la même apparence et reçoit seulement son texte ou son action.

Composants

Navbar et navigation responsive

Navbar choisit entre une navigation mobile et une navigation ordinateur. Les liens sont définis dans MenuNav afin d'éviter de recopier les mêmes adresses dans plusieurs composants.

Dans le fichier `src/components/navbar/Navbar.jsx`

```
export default function Navbar() {
  return (
    <header className="w-full">
      <NavMobile />
      <NavWeb />
    </header>
  );
}
```

NavMobile et NavWeb existent toutes les deux dans le code, mais Tailwind décide laquelle est visible selon la largeur. La version ordinateur est cachée avant le palier lg. La version mobile est affichée à sa place.

Dans le fichier `src/components/navbar/MenuNav.jsx`

```
const location = useLocation();

const linkClass = (path) => {
  const isActive = location.pathname === path;
  return isActive
    ? "text-[#EFBB6F] underline underline-offset-4"
    : "text-amber-50 hover:text-[#EFBB6F]";
};

<Link to="/tirage-gratuit"
      className={linkClass("/tirage-gratuit")}
      onClick={closeMenu}>
  Tirage gratuit
</Link>
```

useLocation récupère l'adresse actuelle. linkClass compare cette adresse avec le lien. Si les deux correspondent, le lien devient doré et souligné. Sinon, il garde son apparence normale et change au survol.

Link vient de React Router. Il permet d'aller vers une autre page sans recharger tout le site. closeMenu est surtout utile sur mobile : après le clic, le menu se referme automatiquement afin de laisser la place au nouveau contenu.

Pourquoi séparer NavWeb et NavMobile ?

Les deux navigations n'ont pas la même disposition. Sur ordinateur, les liens et les boutons peuvent rester visibles. Sur téléphone, l'espace est limité et un menu ouvrant est plus adapté. Garder deux présentations me donne plus de contrôle, tout en partageant le même MenuNav.

Maintenance : Ajouter ou modifier un lien dans MenuNav met à jour la navigation partout où ce composant est utilisé.

Interfaces statiques

HomeCard : créer une carte réutilisable

La page d'accueil contient plusieurs cartes qui ont la même structure, mais pas le même texte. Au lieu d'écrire cinq fois le même bloc, j'ai créé HomeCard. Il reçoit un titre, une description et une icône.

Dans le fichier `src/components/deco/HomeCard.jsx`

```
export default function HomeCard({ title, description, icon }) {
  return (
    <article className="rounded-2xl border border-[#EFBB6F]
      bg-[#070F24]/75 px-6 py-6 text-center">
      <img src={icon} alt="" className="mx-auto h-24 w-24" />
      <h3 className="text-xl text-[#EFBB6F]">{title}</h3>
      <p className="italic text-white">{description}</p>
    </article>
  );
}
```

Les props

title, description et icon sont des props. Ce sont des informations envoyées au composant au moment où je l'utilise. La structure de la carte reste identique, mais le contenu peut changer.

Dans le fichier `src/components/pages/home/HomeCards.jsx`

```
<HomeCard
  icon="../images/runes_1.svg"
  title="Tirage gratuit"
  description="Obtenez 3 runes et leur interprétation immédiate"
/>

<HomeCard
  icon="../images/book.svg"
  title="Répertoire des runes"
  description="Explorez la signification des 24 runes"
/>
```

Si je change la bordure, la taille du titre ou l'espace autour de l'icône dans HomeCard, toutes les cartes sont mises à jour. HomeCards s'occupe ensuite de leur disposition : une colonne sur téléphone, deux sur tablette et davantage sur les grands écrans.

Pourquoi cette page représente une interface statique

Au départ, les textes et les icônes de l'accueil étaient écrits directement dans le front-end. Cette étape m'a permis de valider la hiérarchie, l'ambiance, les boutons et le responsive avant de relier les autres écrans aux données réelles.

Définition à retenir : Un composant est un élément réutilisable de l'interface. Les props permettent de réutiliser sa structure avec des contenus différents.

Interfaces dynamiques

États React et déroulement d'un tirage

Une interface devient dynamique lorsqu'elle réagit à une action ou à une donnée. Dans React, `useState` me permet de mémoriser une valeur : la question saisie, la catégorie, la liste des runes, le message d'erreur ou l'état de chargement.

Dans le fichier `src/pages/membres/TirageGratuit.jsx`

```
async function handleStartDraw() {
  setMessage("");
  setIsLoadingDraw(true);
  setShowDescriptions(false);
  setDrawnRunes([]);

  try {
    const response = await createNornesDraw(question, category);

    if (!response.success) {
      setMessage(response.message || "Erreur lors du tirage.");
      return;
    }

    const formattedDraw = response.draw.map((item) => ({
      position: item.position,
      symbole: item.rune.symbole,
      nom: item.rune.nom,
      etat: item.rune.etat,
      interpretation: item.interpretation,
    }));

    setDrawData(formattedDraw);
    setDrawStarted(true);
  } finally {
    setIsLoadingDraw(false);
  }
}
```

Lecture du code étape par étape

Je commence par vider les anciens messages et les anciennes runes. `setIsLoadingDraw(true)` indique qu'une demande est en cours. La fonction `createNornesDraw` envoie la question et la catégorie. `await` demande au code d'attendre la réponse avant de continuer. Si `success` vaut `false`, j'affiche le message reçu et j'arrête la fonction avec `return`. Si la demande réussit, `map` parcourt les résultats et prépare un objet plus simple pour chaque rune. Je conserve seulement les informations utiles à l'affichage.

`setDrawData` enregistre les runes préparées. `setDrawStarted(true)` autorise ensuite le composant d'animation à commencer. Le bloc `finally` est toujours exécuté, même lorsqu'une erreur arrive : il remet le chargement à `false`.

Les autres parties dynamiques

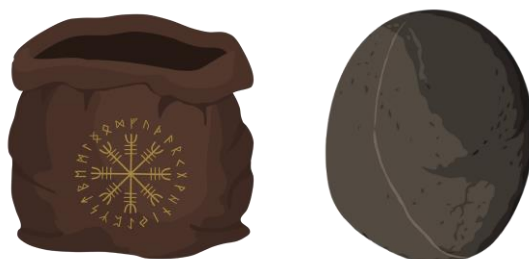
Le glossaire charge les runes et applique des filtres. Les formulaires gardent les valeurs saisies et affichent des messages. Les tableaux de bord chargent leurs listes puis les mettent à jour après une suppression ou une modification.

Le trajet général : Action de l'utilisateur → appel du service → réponse reçue → état React modifié → écran mis à jour.

Animations

Faire vivre le tirage sans perdre la logique

L'animation du tirage est importante dans mon projet, mais elle ne doit pas mélanger toutes les responsabilités. DrawAnimation gère le déroulement visuel. La page TirageGratuit garde les données et décide quand afficher les interprétations.



Le sac et la pierre sont utilisés par des composants séparés.

Dans le fichier `src/components/draws/DrawAnimation.jsx`

```
setIsShaking(true);

setTimeout(() => {
  setIsShaking(false);
  setFlyingRune(selectedRune);
  setFlyingStep("out");

  setTimeout(() => setFlyingStep("pause"), 300);
  setTimeout(() => setFlyingStep("slide"), 650);
  setTimeout(() => setFlyingStep("flip"), 1200);

  setTimeout(() => {
    setDrawnRunes((current) => [...current, selectedRune]);
    setFlyingRune(null);
    setFlyingStep(null);
  }, 1700);
}, 500);
```

Les états utilisés

`isShaking` indique si le sac doit bouger. `flyingRune` contient la pierre actuellement animée. `flyingStep` suit son étape : sortie, pause, déplacement et retournement. `drawnRunes` contient les runes déjà révélées.

Les `setTimeout` déclenchent les étapes avec un délai. À la fin, la nouvelle rune est ajoutée à `drawnRunes` avec une copie du tableau précédent. Les états temporaires reviennent ensuite à `null` pour préparer le clic suivant.

Éviter les doubles clics et guider la lecture

Avant de commencer, je vérifie qu'aucune pierre n'est déjà en mouvement et que le tirage n'est pas terminé. Cela empêche deux animations de se lancer en même temps. Après la dernière rune, j'attends la fin visuelle puis `scrollIntoView` conduit doucement l'utilisateur vers les interprétations.

Le Cercle d'Odin

Le tirage à neuf runes utilise le même principe, avec un placement en cercle sur grand écran. Le calcul transforme chaque position en angle, puis utilise le cosinus et le sinus pour obtenir les coordonnées. Sur mobile, le cercle devient une grille de trois colonnes, plus simple à lire.

Formulaires

Connexion : séparer la page et le formulaire

Pour la connexion, j'ai séparé l'affichage, la logique et l'appel vers l'API. Quatre fichiers travaillent ensemble : Connexion.jsx, LoginForm.jsx, UseLogin.jsx et AuthService.jsx. Cette organisation me permet de savoir rapidement où intervenir.

Fichier	Rôle
Connexion.jsx	Place le fond et le formulaire dans la page
LoginForm.jsx	Affiche les champs, boutons et messages
UseLogin.jsx	Valide les valeurs et choisit la redirection
AuthService.jsx	Envoie la demande et conserve la connexion

Dans le fichier `src/pages/public/Connexion.jsx`

```
export default function Connexion() {
  const login = useLogin();

  return (
    <main className="relative min-h-[calc(100vh-80px)]">
      <section className="flex min-h-[calc(100vh-80px)]
        items-center justify-center px-6 py-12">
        <LoginForm login={login} />
      </section>
    </main>
  );
}
```

useLogin rassemble les valeurs et les fonctions nécessaires. La variable login contient notamment l'e-mail, le mot de passe, les messages, les fonctions de mise à jour et handleSubmit. La page transmet cet ensemble au formulaire avec la prop login.

Dans le fichier `src/components/forms/LoginForm.jsx`

```
<form onSubmit={login.handleSubmit}>
  <input
    type="email"
    value={login.email}
    onChange={(event) => login.setEmail(event.target.value)}
  />

  <input
    type="password"
    value={login.password}
    onChange={(event) => login.setPassword(event.target.value)}
  />

  {login.error && <p>{login.error}</p>}
  <ButtonGold type="submit" text="Me connecter" />
</form>
```


Les champs sont contrôlés par React : leur valeur vient du hook et chaque saisie appelle la fonction correspondante. `onSubmit` lance `handleSubmit`. Le formulaire affiche le message reçu, mais il ne contient pas lui-même toutes les vérifications.

Formulaires

Connexion : validation, redirection et service

Le hook `UseLogin`

Le hook mémorise les champs et les messages avec `useState`. Au moment de l'envoi, `handleSubmit` empêche le rechargement de la page, nettoie l'e-mail puis vérifie chaque règle avant d'appeler le service.

Dans le fichier `src/hooks/UseLogin.jsx`

```
async function handleSubmit(event) {
  event.preventDefault();
  setError("");

  const cleanEmail = email.trim().toLowerCase();

  if (!isValidEmail(cleanEmail)) {
    setError("L'email n'est pas valide.");
    return;
  }

  if (password.length < 8) {
    setError("Le mot de passe doit contenir au moins 8 caractères.");
    return;
  }

  const data = await loginUser(cleanEmail, password);

  if (!data.success) {
    setError(data.message || "Identifiants incorrects.");
    return;
  }

  saveAuth(data.user, data.token);
  navigate(data.user.role === "admin"
    ? "/admin" : "/dashboard-membre", { replace: true });
}
```

`trim` retire les espaces inutiles et `toLowerCase` met l'adresse en minuscules.

Chaque erreur arrête la fonction avec `return`. Si la connexion réussit, `saveAuth` enregistre l'utilisateur et le jeton, puis `navigate` envoie la personne vers l'espace correspondant à son rôle.

Le service `AuthService`

Dans le fichier `src/services/AuthService.jsx`

```
export async function loginUser(email, password) {
  const response = await fetch(`${API_URL}/api/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });

  return await response.json();
}

export function saveAuth(user, token) {
  localStorage.setItem("user", JSON.stringify(user));
  localStorage.setItem("token", token);
}
```

fetch envoie une requête POST vers /api/login. JSON.stringify transforme les valeurs JavaScript en texte JSON. La réponse est ensuite reconvertie avec response.json(). localStorage conserve la connexion dans le navigateur afin de la retrouver lors d'un changement de page.

Pour les demandes protégées, authFetch récupère le jeton et l'ajoute dans l'en-tête Authorization. Cette fonction évite de répéter la même préparation dans chaque service. Si le serveur répond 401, la connexion locale est supprimée et une redirection peut être demandée.

Séparation obtenue : La page place le formulaire, le formulaire affiche, le hook vérifie et redirige, le service communique avec l'API.

Espace membre

Dashboard membre et historique des tirages

Le tableau de bord membre permet de retrouver les tirages enregistrés et de gérer son compte. Il se trouve dans MemberLayout : une barre latérale est utilisée sur ordinateur et une navigation inférieure prend sa place sur mobile.

Dans le fichier src/pages/membres/DashboardMembre.jsx

```
const [results, setResults] = useState([]);
const [message, setMessage] = useState("");
const [loading, setLoading] = useState(true);

async function loadResults() {
  setLoading(true);
  const response = await getMemberResults();

  if (response.success) {
    setResults(response.results);
  } else {
    setMessage(response.message);
  }

  setLoading(false);
}

useEffect(() => {
  loadResults();
}, []);
```

results contient la liste des tirages, message le retour destiné à l'utilisateur et loading indique si la demande est en cours. useEffect appelle loadResults au premier affichage. Son tableau vide signifie que cette action n'est lancée qu'une fois au montage du composant.

Supprimer un résultat sans recharger toute la page

Dans le fichier `src/pages/membres/DashboardMembre.jsx`

```
async function handleDelete(id) {
  const response = await deleteMemberResult(id);

  if (response.success) {
    setResults((currentResults) =>
      currentResults.filter((result) => result.id !== id)
    );
    setMessage("Tirage supprimé avec succès.");
  }
}
```

Après la confirmation du serveur, `filter` crée une nouvelle liste qui ne contient plus l'identifiant supprimé. React reçoit cette nouvelle liste et remet immédiatement l'écran à jour.

Télécharger et présenter les résultats

Le service de téléchargement reçoit le PDF sous forme de blob, crée un lien temporaire, lance le téléchargement puis retire ce lien. Sur téléphone, chaque tirage est présenté comme une carte. À partir du palier md, les mêmes informations sont affichées dans un tableau plus compact.

J'ai aussi prévu les différents cas : chargement en cours, message d'erreur, historique vide et liste disponible. Un tableau de bord ne doit pas rester blanc pendant l'attente ni afficher un tableau vide sans explication.

Ce que cette page réunit : Chargement des données, états d'attente, affichage conditionnel, suppression, téléchargement et présentation responsive.

Espace administrateur

Dashboard administrateur et statistiques

Le dashboard administrateur donne une vue d'ensemble du site. J'ai séparé chaque bloc dans un composant différent afin de retrouver facilement le code lié aux statistiques, aux derniers membres, aux derniers tirages ou aux runes.

Dans le fichier `src/pages/admin/DashboardAdmin.jsx`

```
export default function DashboardAdmin() {
  const user = getUser();

  return (
    <div className="space-y-6">
      <header className="rounded-2xl border px-6 py-5">
        <h1>Bienvenue, {user?.firstname || "Admin"}</h1>
      </header>

      <AdminStats />
      <div className="grid gap-6 xl:grid-cols-2">
        <LastMembers />
        <LastDraws />
      </div>
      <RunesAdmin />
    </div>
  );
}
```

La page récupère l'utilisatrice connectée avec `getUser`. L'écriture `user?.firstname` vérifie que `user` existe avant de chercher le prénom. Si le prénom n'est pas disponible, le texte Admin est affiché.

AdminStats occupe toute la largeur. LastMembers et LastDraws passent sur deux colonnes à partir du palier xl. RunesAdmin donne accès à la gestion complète des runes. DashboardAdmin sert donc surtout à assembler les blocs ; chaque composant s'occupe ensuite de ses propres données.

Chargement des statistiques

Dans le fichier `src/components/pages/admin/AdminStats.jsx`

```
const [adminStats, setAdminStats] = useState({
  runes: 0,
  draws: 0,
  users: 0,
  active_runes: 0,
});

useEffect(() => {
  async function loadStats() {
    const response = await getAdminStats();
    if (response.success) setAdminStats(response);
  }
  loadStats();
}, []);
```

Les valeurs commencent à zéro afin que les cartes puissent déjà s'afficher pendant l'attente. Au premier rendu, `loadStats` demande les chiffres. Si la réponse réussit, `setAdminStats` remplace les valeurs de départ par les données reçues.

Les autres blocs

LastMembers présente les derniers comptes créés. LastDraws montre les derniers tirages. L'administration possède également des interfaces dédiées pour consulter les utilisateurs, modifier un rôle, supprimer un compte, télécharger un PDF ou retirer un résultat.

Choix de structure : La page principale reste lisible parce qu'elle assemble des composants spécialisés au lieu de contenir toute l'administration dans un seul fichier.

Administration

Gestion des runes : créer, modifier et actualiser

La gestion des runes est la partie la plus complète de l'administration. Trois éléments travaillent ensemble : RunesManagement coordonne l'écran, RuneForm gère le formulaire et RunesTable affiche les runes existantes.

Dans le fichier `src/pages/admin/RunesManagement.jsx`

```
const [showForm, setShowForm] = useState(false);
const [runeToEdit, setRuneToEdit] = useState(null);
const [refreshTable, setRefreshTable] = useState(false);

async function handleEdit(id) {
  const response = await getRuneById(id);

  if (response.success) {
    setRuneToEdit({
      ...response.rune,
      interpretations: response.interpretations,
    });
    setShowForm(true);
  }
}
```

`showForm` indique si le formulaire est visible. `runeToEdit` vaut `null` pour une création et contient une rune pour une modification. `refreshTable` change après une action réussie afin de demander au tableau de charger la liste à jour.

Un seul formulaire pour deux actions

Dans le fichier `src/components/forms/RuneForm.jsx`

```
const isEditing = Boolean(runeToEdit);
const dataToSend = { rune, interpretations };

const response = isEditing
  ? await updateRune(runeToEdit.id, dataToSend)
  : await createRune(dataToSend);

if (response.success && onSuccess) {
  onSuccess();
}
```

`Boolean` transforme la présence de `runeToEdit` en `true` ou `false`. En modification, le formulaire est prérempli et appelle `updateRune`. En création, les champs sont vides et `createRune` est utilisé. Le formulaire est découpé en étapes : informations principales, interprétations par catégorie puis récapitulatif.

Actualiser après l'enregistrement

Dans le fichier `src/components/tables/RunesTable.jsx`

```
useEffect(() => {
  loadRunes();
}, [refreshTable]);
async function handleDelete(id) {
  const confirmed = window.confirm(
    "Voulez-vous supprimer cette rune ?" );
  if (!confirmed) return;
  const response = await deleteRune(id);
  setMessage(response.message);
  if (response.success) loadRunes();
}
```

Après une création ou une modification, `RunesManagement` ferme le formulaire, vide `runeToEdit` et inverse `refreshTable`. Comme `useEffect` surveille cette valeur, `loadRunes` se relance. Pour la suppression, `window.confirm` évite une erreur de clic, puis la liste est rechargée seulement si le serveur confirme la réussite.

Sur mobile, une rune est présentée sous forme de carte avec ses actions. Sur grand écran, les mêmes données prennent la forme d'un tableau classique.

Web mobile

Responsive design avec Tailwind CSS

J'ai prévu le responsive dès les maquettes, puis je l'ai construit avec les paliers de Tailwind. Une classe sans préfixe s'applique d'abord au mobile. Les préfixes sm, md, lg et xl ajoutent ensuite des règles quand l'écran devient plus large.

Dans le fichier `src/components/pages/home/HomeVideo.jsx`

```
{/* MOBILE */}
<div className="absolute inset-0 block md:hidden">
  
</div>

{/* TABLETTE ET ORDINATEUR */}
<div className="absolute inset-0 hidden md:block">
  <video autoPlay muted loop playsInline
    className="h-full w-full object-cover">
    <source src="../../../videos/video_site.mp4"
      type="video/mp4" />
  </video>
</div>
```

`block md:hidden` affiche l'image avant le palier `md` puis la cache. `hidden md:block` fait l'inverse pour la vidéo. Je garde donc une image plus légère sur téléphone et l'animation complète à partir de la tablette.

Adaptations principales

Élément	Mobile	Écran large
Navigation publique	Menu ouvrant	Barre complète
Accueil	Image fixe	Vidéo d'ambiance
HomeCards	Une colonne	Deux puis cinq colonnes
Historique membre	Cartes	Tableau
Administration	Cartes et navigation réduite	Tableaux et sidebar
Cercle d'Odin	Grille de 3 colonnes	Placement circulaire

Le responsive ne consiste donc pas seulement à réduire la taille des textes. J'ai parfois changé la structure complète : un tableau devient une série de cartes, une sidebar devient un menu inférieur et un cercle devient une grille.

Vérifier le résultat sur plusieurs largeurs

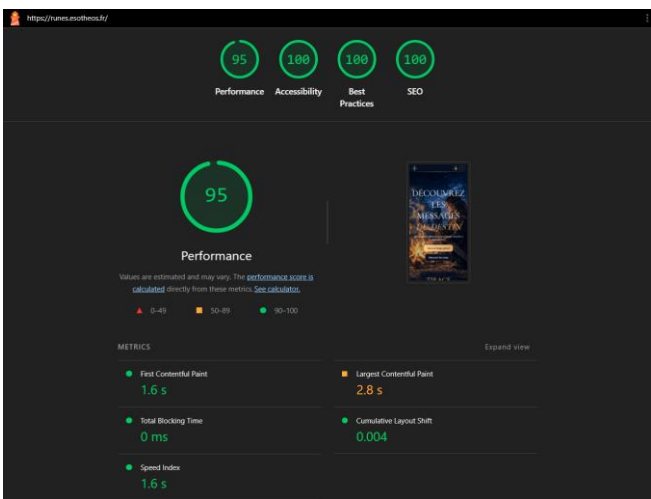
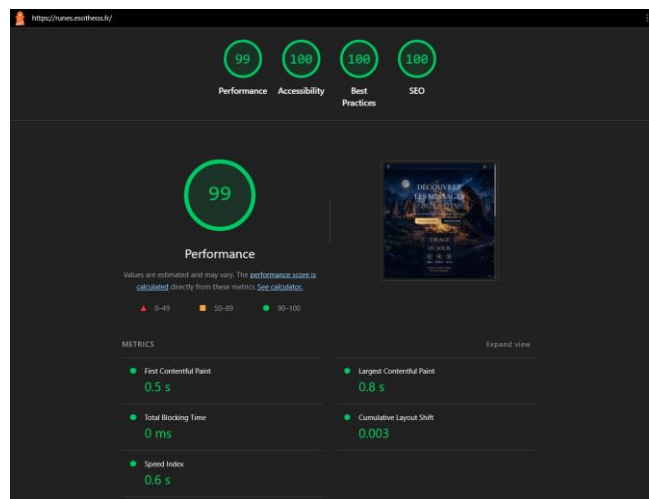
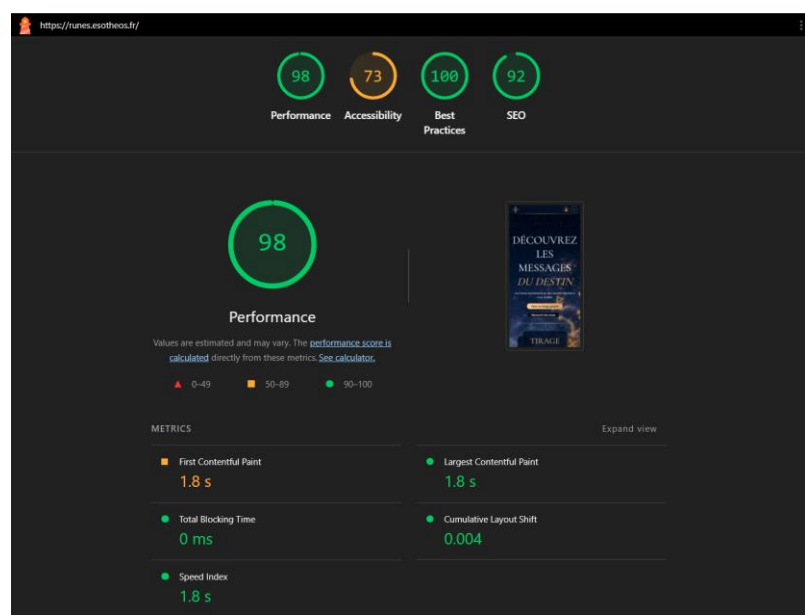
Après ces adaptations, j'ai contrôlé le site sur plusieurs tailles d'écran. J'ai ensuite utilisé Lighthouse pour mesurer quatre points : les performances, l'accessibilité, les bonnes pratiques et le référencement naturel. Cet audit m'a permis de passer d'un contrôle visuel à une vérification plus précise.

Étape suivante : J’ai commencé par relever les problèmes dans l’audit initial, puis j’ai corrigé le code et relancé les mesures sur ordinateur et sur mobile.

Contrôle du site

Audit Lighthouse : avant et après les corrections

Une fois le site en ligne, j’ai lancé un premier audit Lighthouse sur <https://runes.esotheos.fr/>. Lighthouse est un outil intégré aux navigateurs basés sur Chromium. Il analyse une page et donne des indications sur ses performances, son accessibilité, les bonnes pratiques et son référencement naturel.



De gauche à droite : audit initial, résultat final sur ordinateur et résultat final sur mobile.

Mesure	Avant	Après ordinateur	Après mobile
Performance	98	99	95

Accessibilité	73	100	100
Bonnes pratiques	100	100	100
SEO	92	100	100

Comment j'interprète ces résultats

Le passage de 73 à 100 en accessibilité confirme que les corrections de navigation, de clavier et de description des icônes ont été reconnues par l'outil. Le SEO passe de 92 à 100 après l'ajout des titres, des descriptions et des fichiers d'indexation.

La performance varie de 95 à 99 selon le type d'audit. Ce résultat est normal : Lighthouse simule des conditions différentes et les mesures peuvent changer selon l'appareil, le réseau ou le moment du test. Les deux valeurs restent dans la zone verte.

Ma démarche : J'ai lu les problèmes détaillés, traité en priorité l'accessibilité, puis le SEO et le chargement des médias. Après chaque série de modifications, j'ai remis le site en ligne et relancé Lighthouse.

Accessibilité

Rendre la navigation plus compréhensible et utilisable au clavier

Le premier audit m'a surtout amenée à revoir la navigation. Plusieurs boutons utilisaient uniquement une icône. Visuellement, leur rôle me paraissait évident, mais un lecteur d'écran ne peut pas deviner la signification d'un dessin. J'ai donc ajouté des informations textuelles dans le code.

Décrire le bouton du menu mobile

Dans le fichier `src/components/navbar/ButtonsNavMobile.jsx`

```
<button
  type="button"
  onClick={() => setOpenMenu(!openMenu)}
  aria-controls={menuId}
  aria-expanded={openMenu}
  aria-label={
    openMenu ? "Fermer le menu principal" : "Ouvrir le menu principal"
  }
  className="focus-visible:outline-2
            focus-visible:outline-offset-2
            focus-visible:outline-white"
>
  <svg aria-hidden="true" focusable="false">...</svg>
</button>
```

`aria-label` donne un nom compréhensible au bouton. `aria-expanded` indique si le menu est ouvert ou fermé. `aria-controls` relie le bouton à la zone du menu grâce à son identifiant. L'icône SVG devient décorative avec `aria-hidden`, car le texte accessible est déjà fourni par le bouton.

Indiquer la page active

Dans le fichier `src/components/navbar/MenuNav.jsx`

```
<nav aria-label="Navigation principale">
  <Link
    to="/glossaire-des-runes"
    aria-current={
      location.pathname === "/glossaire-des-runes"
        ? "page"
        : undefined
    }
  >
    Glossaire des runes
  </Link>
</nav>
```

La balise `nav` précise qu'il s'agit de la navigation principale. `aria-current="page"` indique au lecteur d'écran quel lien correspond à la page affichée. Visuellement, ce lien reste aussi doré et souligné.

Aller directement au contenu

J'ai ajouté un lien « Aller au contenu principal ». Il reste caché tant que l'utilisateur ne navigue pas au clavier. Lorsqu'il reçoit le focus, il devient visible et permet de passer directement la barre de navigation. Les trois layouts possèdent maintenant un élément avec l'identifiant `contenu-principal` et `tabIndex="-1"`.

Améliorer les boutons

Les boutons administrateur ont maintenant une zone minimale de 48 par 48 pixels avec `min-h-12` et `min-w-12`. J'ai ajouté un contour visible au clavier avec `focus-visible`, ainsi qu'un état disabled plus clair. Les icônes décoratives sont ignorées par les lecteurs d'écran, tandis que le texte du bouton reste disponible.

Ce que j'ai compris : Une interface accessible ne dépend pas seulement des couleurs. Il faut aussi décrire les actions, rendre le focus visible et conserver une navigation logique au clavier.

Performances

Alléger le chargement sans supprimer l'univers visuel

Mon accueil utilise une image forte sur mobile et une vidéo sur ordinateur. Je ne voulais pas supprimer cette ambiance, mais éviter de télécharger les ressources les plus lourdes lorsqu'elles ne sont pas nécessaires. J'ai donc adapté les médias selon la largeur de l'écran et leur importance dans la page.

Charger les pages seulement lorsqu'elles sont demandées

Dans le fichier `src/main.jsx`

```
const GlossaireDesRunes = lazy(
  () => import("../pages/public/GlossaireDesRunes")
);
const DashboardMembre = lazy(
  () => import("../pages/membres/DashboardMembre")
);

<Suspense fallback={routeFallback}>
  <Routes>...</Routes>
</Suspense>
```

`lazy` sépare les pages du fichier principal. Le navigateur charge une page lorsqu'elle est réellement demandée. `Suspense` affiche un fond temporaire pendant ce court chargement. L'accueil reste importé normalement, car c'est la première page affichée.

Choisir une image adaptée à l'écran

Dans le fichier `src/components/deco/ResponsiveBackgroundImage.jsx`

```
<picture>
  <source media={mobileMedia} srcSet={mobileSrc} />
  <img
    src={desktopSrc}
    alt={alt}
    width={width}
    height={height}
    decoding="async"
    className={className}
  />
</picture>
```

J'ai créé des images de 900 pixels pour les petits écrans et des versions de 1600 pixels pour les écrans plus larges. `picture` et `source` permettent au navigateur de choisir le fichier adapté. Les dimensions `width` et `height` réservent la place avant la fin du chargement et limitent les déplacements de la mise en page.

Donner la priorité au visuel principal

L'image principale de l'accueil possède plusieurs tailles, un `srcSet` et la priorité `high`, car elle est visible immédiatement. Elle est aussi préchargée dans `Helmet`. À l'inverse, la vidéo de bureau utilise `preload="none"` et n'est affichée qu'à partir de 1024 pixels.

Optimiser les polices et les médias

J'ai ajouté `preconnect` vers Google Fonts pour commencer plus tôt la connexion aux serveurs de polices. La feuille de styles est d'abord chargée sans bloquer l'affichage, puis activée. Les images décoratives utilisent un texte alternatif vide ou `aria-hidden` ; les autres conservent une description utile.

Le temps de blocage reste à 0 ms dans les trois captures. Le score final de performance atteint 99 sur l'audit ordinateur et 95 sur l'audit mobile.

Mon choix : Conserver une page immersive, mais envoyer au navigateur le bon fichier, au bon moment et pour le bon écran.

Référencement naturel

Helmet Async, sitemap et robots.txt

Une application React affiche plusieurs pages dans un seul document HTML. Sans gestion supplémentaire, elles risquent de partager le même titre et la même description. J'ai installé react-helmet-async dans sa version exacte 3.0.0 afin de modifier les informations de l'en-tête selon la page affichée.

Mettre en place HelmetProvider

Dans le fichier `src/main.jsx` et `src/pages/public/Accueil.jsx`

```

<HelmetProvider>
  <BrowserRouter>
    <Routes>...</Routes>
  </BrowserRouter>
</HelmetProvider>

<Helmet>
  <title>Accueil - Runes et Destin</title>
  <meta
    name="description"
    content="Découvrez la sagesse des runes nordiques à travers
              des tirages en ligne et des interprétations détaillées."
  />
</Helmet>

```

HelmetProvider entoure l'application et autorise les composants Helmet. J'ai ensuite ajouté un titre et une description adaptés à l'accueil, au glossaire, à la connexion, à l'inscription, aux tirages et à plusieurs pages membres. Le moteur de recherche reçoit ainsi une présentation plus précise du contenu affiché.

Le fichier sitemap.xml

Dans le fichier `public/sitemap.xml`

```

<url>
  <loc>https://runes.esotheos.fr/tirage-du-jour</loc>
  <changefreq>daily</changefreq>
  <priority>0.9</priority>
</url>

<url>
  <loc>https://runes.esotheos.fr/glossaire-des-runes</loc>
  <changefreq>monthly</changefreq>
  <priority>0.8</priority>
</url>

```

Le sitemap liste les pages publiques que je souhaite faire découvrir aux moteurs : l'accueil, le tirage du jour, le glossaire, une page de rune et la page À propos. changefreq indique une fréquence indicative de modification et priority leur importance relative dans le site.

Le fichier robots.txt

Dans le fichier public/robots.txt

```
User-agent: *
Allow: /

Disallow: /dashboard-membre
Disallow: /admin
Disallow: /panier
Disallow: /inscription
Disallow: /connexion

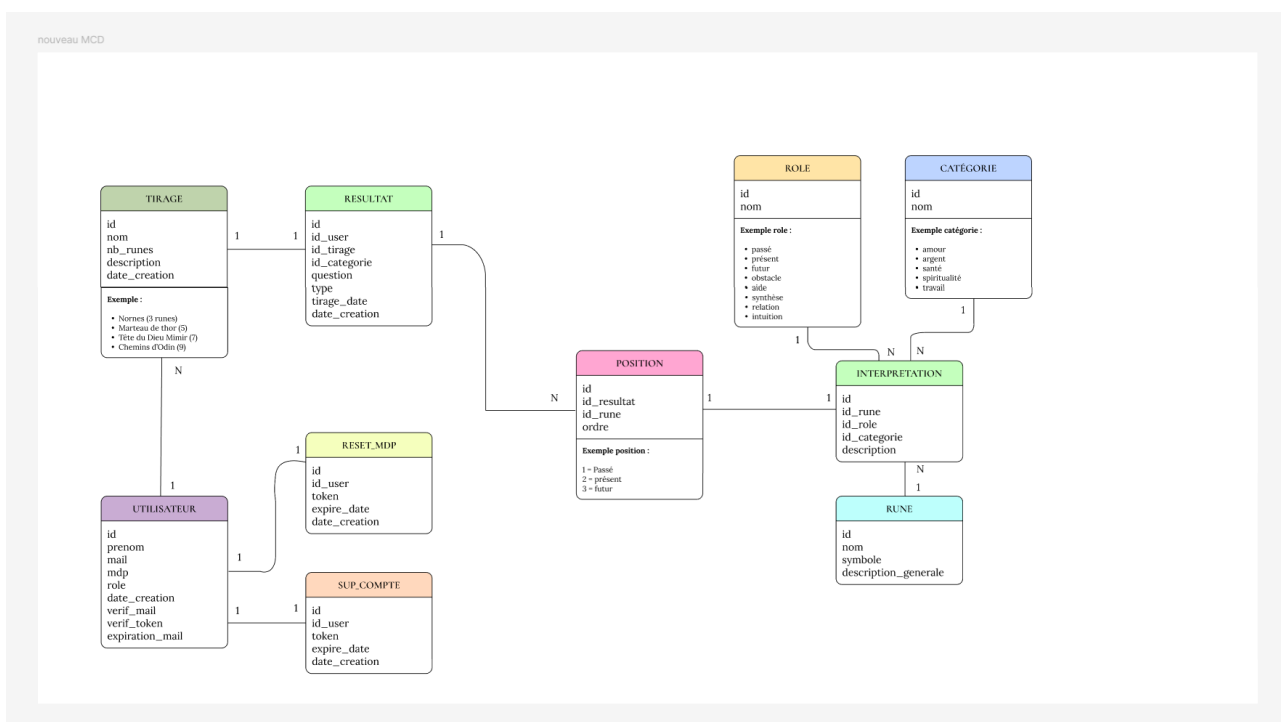
Sitemap: https://runes.esotheos.fr/sitemap.xml
```

robots.txt autorise l'exploration générale, mais demande de ne pas parcourir les espaces personnels, l'administration et certaines pages qui n'ont pas d'intérêt dans les résultats de recherche. La dernière ligne indique directement l'adresse du sitemap.

Conception du back-end

Modèle de données : le MCD

Avant d'écrire les requêtes SQL, j'ai représenté les besoins de l'application sous la forme d'un modèle conceptuel de données. Le MCD décrit les informations utiles et leurs liens sans dépendre encore de MySQL. Il m'a aidée à vérifier qu'un tirage pouvait être retrouvé, relié à ses runes et interprété selon son contexte.



MCD stabilisé après le retrait de la vente en ligne de la première version.

L'entité UTILISATEUR représente le compte. Un RESULTAT correspond à un tirage enregistré. Les POSITIONS associent ce résultat aux runes sorties et conservent leur ordre. Une INTERPRETATION relie une rune, un rôle et une catégorie afin de fournir le texte adapté.

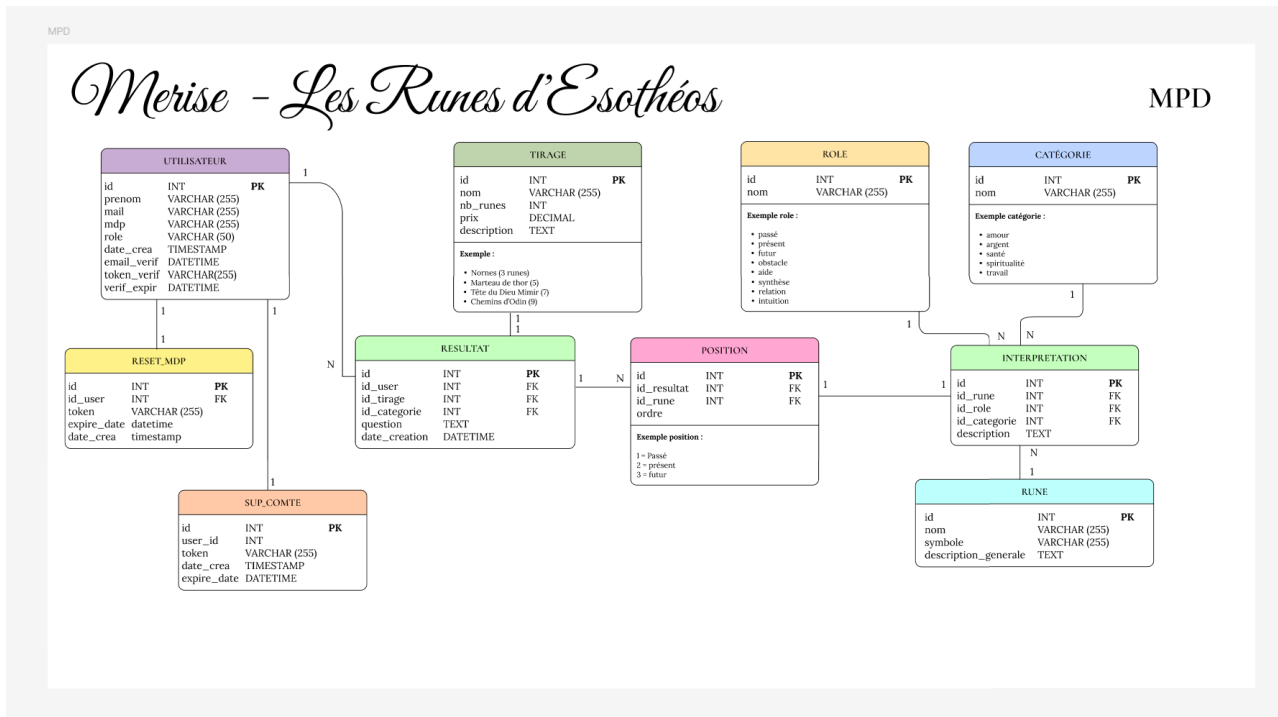
Le modèle a évolué pendant le développement. Les tables liées au paiement ont été retirées du périmètre de l'examen. À l'inverse, les demandes de réinitialisation du mot de passe et de suppression de compte ont été ajoutées pour couvrir tout le cycle de vie d'un compte.

Cardinalité importante : Un résultat contient plusieurs positions, mais chaque position appartient à un seul résultat. Cette table d'association permet de garder l'ordre exact des runes tirées.

Conception du back-end

Du MLD au modèle physique

Le modèle logique transforme les entités en relations. Les identifiants deviennent des clés primaires et les liens deviennent des clés étrangères. Le modèle physique ajoute ensuite les types MySQL et les contraintes nécessaires à l'implémentation.



Passage des relations aux tables, avec les clés primaires et étrangères.

Dans mes premiers schémas, les noms étaient en français. Dans le code final, j'ai choisi des noms anglais cohérents : users, draws, results, positions et interpretations. La logique reste identique. Cette évolution explique les petites différences de vocabulaire entre le diagramme de conception et les requêtes du projet.

Dans le fichier schéma relationnel simplifié

```

RESULTS (#id, question, type, draw_date,
  id_user -> USERS.id,
  id_draw -> DRAWS.id,
  id_category -> CATEGORIES.id)
POSITIONS (#id, ordre,
  id_result -> RESULTS.id,
  id_rune -> RUNES.id)
INTERPRETATIONS (#id, etat, description,
  id_rune -> RUNES.id,
  id_role -> ROLES.id,
  id_category -> CATEGORIES.id)
  
```

Lecture : Le symbole # représente une clé primaire. La flèche indique la table et la colonne visées par une clé étrangère.

Conception du back-end

Dictionnaire des données — cœur fonctionnel

Le dictionnaire précise ce que contient chaque table et la raison de sa présence. Les tailles détaillées restent gérées dans MySQL ; je présente ici les types utiles pour comprendre la structure.

Table	Champs principaux	Types / contraintes	Rôle
users	id, firstname, email, password, role	INT, VARCHAR ; email unique	Compte, identité et droits
draws	id, name, nbrunes, description	INT, VARCHAR, TEXT	Formats de tirage disponibles
runes	id, nom, symbole, aettir, etat	INT, VARCHAR ; état contrôlé	Données générales d'une rune
categories	id, nom	INT, VARCHAR	Domaine : amour, travail...
roles	id, nom	INT, VARCHAR	Fonction d'une position
interpretations	id, idrune, idrole, idcategory, etat, description	FK + TEXT	Texte choisi selon le contexte

Pourquoi séparer rôles et catégories ?

Une même rune n'a pas la même lecture dans tous les cas. La catégorie répond au domaine choisi par l'utilisateur. Le rôle dépend de la place de la rune dans le tirage : passé, présent, futur, obstacle, aide ou synthèse. La table interpretations combine ces informations avec l'état droit ou renversé.

Éviter les répétitions

Les informations générales d'une rune ne sont enregistrées qu'une fois dans runes. Les interprétations sont stockées séparément. Cette normalisation évite de recopier le nom, le symbole et la description générale pour chaque catégorie et chaque position.

Bénéfice : Une nouvelle catégorie peut être ajoutée sans modifier la structure d'un résultat ni dupliquer toutes les informations générales des runes.

*Conception du back-end***Dictionnaire et schéma relationnel**

Table	Champs principaux	Contraintes utiles	Usage
results	id, iduser, iddraw, idcategory, question, type, drawdate	iduser nullable ; FK	Tirage membre ou quotidien
positions	id, idresult, idrune, ordre	FK ; ordre numérique	Runes d'un résultat dans l'ordre
passwordresets	id, userid, token, expiresat	FK ; jeton haché ; 1 h	Réinitialisation du mot de passe
accountdeletions	id, userid, token, expiresat	FK ; jeton haché ; 1 jour	Confirmation de suppression

Relations essentielles

- USERS 1,N RESULTS : un membre peut conserver plusieurs tirages ; un tirage quotidien n'a pas d'utilisateur.
- DRAWS 1,N RESULTS et CATEGORIES 1,N RESULTS : chaque résultat garde son format et son domaine.
- RESULTS 1,N POSITIONS et RUNES 1,N POSITIONS : positions matérialise la composition du tirage.
- RUNES, ROLES et CATEGORIES 1,N INTERPRETATIONS : chaque texte correspond à une combinaison précise.
- USERS 1,N PASSWORD_RESETS / ACCOUNT_DELETIONS : les demandes temporaires restent liées au compte.

Intégrité attendue

Les clés étrangères empêchent une position de pointer vers un résultat ou une rune inexistante. L'ordre permet de reconstruire le tirage de façon stable. Le type distingue les résultats user des résultats daily, tandis que draw_date garantit qu'un même tirage du jour peut être réutilisé avant son renouvellement.

Point de vigilance : La création d'un résultat et de toutes ses positions pourrait encore être enveloppée dans une transaction SQL, afin d'annuler l'ensemble si une insertion intermédiaire échoue

*Conception du back-end***Sécurité de la base de données**

La sécurité de la base repose d'abord sur la manière dont l'API s'y connecte et construit ses requêtes. Le navigateur ne communique jamais directement avec MySQL : il passe par les routes du back-end.

Dans le fichier `src/config/database.php`

```
return new PDO(
    'mysql:host=' . $_ENV['DB_HOST'] .
    ';dbname=' . $_ENV['DB_NAME'] . ';charset=utf8mb4',
    $_ENV['DB_USER'], $_ENV['DB_PASSWORD'],
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false
    ]
);
```

Protection	Mise en œuvre	Risque limité
Requêtes préparées	On utilise <code>prepare()</code> puis <code>execute()</code> avec des paramètres	Évite les injections SQL
Vraies préparations PDO	<code>ATTR_EMULATE_PREPARES = false</code>	Évite que PDO simule les requêtes préparées
Secrets hors du dépôt	Identifiants chargés depuis <code>.env</code>	Évite d'exposer le mot de passe MySQL dans le code
Encodage <code>utf8mb4</code>	Défini dans la chaîne de connexion	Évite les problèmes avec les accents, emojis, caractères spéciaux
Clés étrangères	Relations entre comptes, résultats et runes	Évite les données orphelines ou incohérentes

En production, le compte MySQL doit disposer uniquement des droits nécessaires à l'application. Les sauvegardes doivent être régulières et stockées en dehors du répertoire public. Les messages SQL détaillés sont journalisés côté serveur mais ne sont pas renvoyés à l'utilisateur.

Principe : Même une route réservée à l'administratrice utilise des paramètres préparés : le rôle ne rend jamais une saisie fiable par défaut.

Développement back-end

API REST et architecture MVC

J'ai développé l'API en PHP avec FlightPHP. Le point d'entrée `public/index.php` charge les dépendances, les variables d'environnement, les en-têtes CORS et les fichiers de routes. Chaque requête suit ensuite une chaîne claire.

Dans le fichier `chemin d'une requête`

```
Front React
-> route HTTP FlightPHP
-> middleware JWT si nécessaire
-> contrôleur et logique métier
-> modèle PDO et base MySQL
-> réponse JSON avec un statut HTTP
```

Ressource	Exemples de routes	Accès
Authentification	POST /api/register, /api/login	Public
Runes	GET /runes, GET /runes/@id	Public
Tirages	POST /draws/nornes, /draws/odin	Public ou membre
Historique	GET/DELETE /api/member/results/@id	Membre
PDF	GET /api/member/results/@id/pdf	Propriétaire
Administration	PUT/DELETE /runes/@id, /api/admin/users	Admin

Les routes choisissent le contrôleur. Le contrôleur valide la demande, applique la règle métier et construit la réponse. Le modèle centralise les requêtes SQL. Cette séparation évite de mélanger le protocole HTTP, les règles de tirage et l'accès aux données.

Convention REST : GET lit une ressource, POST la crée, PUT la modifie et DELETE la supprime. Les réponses utilisent notamment 201, 400, 401, 403, 404, 409, 429 et 500.

Développement back-end

MVC : de la route au contrôleur

L'architecture MVC sépare trois responsabilités. La route reconnaît l'adresse et la méthode HTTP. Le contrôleur vérifie la demande et décide de l'action. Le modèle dialogue avec MySQL. Cette séparation me permet de retrouver rapidement l'endroit à modifier et d'éviter d'écrire des requêtes SQL directement dans les routes.

La route : le point d'entrée

Dans le fichier `src/routes/runes.php`

```
Flight::route('GET /runes/@id',
    ['RuneController', 'show']);

Flight::route('POST /runes', function () {
    if (!JwtMiddleware::admin()) return;
    RuneController::create();
});
```

La première route transmet l'identifiant au contrôleur. La seconde protège la création avec le middleware admin avant d'appeler la méthode create. La route reste courte : elle ne contient ni validation détaillée ni SQL.

Le contrôleur : validation et réponse HTTP

Dans le fichier `src/controllers/RuneController.php`

```
public static function show($id) {
    $rune = RuneModel::findById((int) $id);

    if (!$rune) {
        Flight::json(['success' => false,
            'message' => 'Rune introuvable.'], 404);
        return;
    }

    Flight::json(['success' => true, 'rune' => $rune]);
}
```

Le contrôleur convertit l'identifiant, appelle le modèle et traduit le résultat en réponse JSON. Il choisit aussi le statut HTTP : 404 si la rune n'existe pas, 200 lorsque la lecture réussit.

Rôle du contrôleur : Il fait le lien entre le langage HTTP du front-end et les méthodes métier du back-end, sans connaître les détails de la connexion PDO.

Développement back-end

MVC : modèle, réponse JSON et CORS

Le modèle : accès aux données

Dans le fichier `src/models/RuneModel.php`

```
public static function findById(int $id): ?array {
    $database = getDatabase();
    $query = $database->prepare(
        'SELECT * FROM runes WHERE id = ?'
    );
    $query->execute([$id]);
    return $query->fetch() ?: null;
}
```

Le modèle reçoit une valeur déjà typée, prépare la requête, transmet l'identifiant séparément et renvoie soit un tableau, soit null. Le contrôleur peut ainsi réutiliser cette méthode sans recopier la requête. Les modèles `DrawModel`, `UserModel` et `MemberResultModel` suivent le même principe.

Le CORS : autoriser le front sans ouvrir l'API à toutes les origines

Dans le fichier `src/config/cors.php`

```
header('Access-Control-Allow-Origin: '
    . $config['frontend_url']);
header('Access-Control-Allow-Methods: '
    . 'GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: '
    . 'Content-Type, Authorization');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit;
}
```

Le front React et l'API étant déployés séparément, le navigateur contrôle leurs échanges. `FRONTEND_URL` fournit l'origine autorisée. Les méthodes et les en-têtes acceptés sont listés explicitement. Pour une requête sensible, le navigateur peut envoyer d'abord une requête `OPTIONS`, appelée *preflight* ; l'API répond 204 avant la vraie requête.

Parcours complet d'une requête

- React envoie la requête avec `Content-Type` et, si nécessaire, `Authorization: Bearer`.
- Le CORS vérifie que l'origine et la méthode font partie de la configuration autorisée.

- FlightPHP sélectionne la route ; le middleware contrôle éventuellement le JWT et le rôle.
- Le contrôleur valide les données, appelle le modèle puis construit la réponse JSON.
- Le modèle exécute une requête PDO préparée et retourne les données au contrôleur.

À ne pas confondre : Le CORS protège les échanges réalisés par le navigateur, mais ne remplace pas l'authentification. Les droits restent contrôlés par le JWT et les middlewares côté serveur.

Développement back-end

JWT et gestion des rôles

Lors de la connexion, le serveur vérifie le mot de passe avec `password_verify`. Si l'adresse e-mail est validée, il crée un JWT signé en HS256. Le jeton contient l'identifiant, l'e-mail, le rôle, sa date de création et une expiration fixée à une heure.

Dans le fichier `src/controllers/AuthController.php`

```
$payload = [
    'id' => $user['id'],
    'email' => $user['email'],
    'role' => $user['role'],
    'iat' => time(),
    'exp' => time() + 3600
];
$token = JWT::encode($payload, getJwtSecret(), 'HS256');
```

Le front transmet ensuite `Authorization: Bearer <token>`. `JwtMiddleware` vérifie la signature et l'expiration, recherche à nouveau l'utilisateur dans la base et compare le rôle du jeton avec le rôle actuel. Une modification de rôle en base est donc prise en compte sans attendre la fin de l'heure.

Niveau	Droits principaux	Contrôle serveur
Visiteur	Glossaire et tirages non sauvegardés	Route publique ou JWT facultatif
Membre	Historique, PDF, mot de passe, suppression du compte	<code>JwtMiddleware::member()</code>
Admin	Utilisateurs, runes, statistiques et résultats	<code>JwtMiddleware::admin()</code>

Défense en profondeur : La route React améliore le parcours, mais le vrai contrôle se trouve dans l'API. Une personne ne peut pas obtenir un droit supplémentaire en modifiant seulement l'affichage du navigateur.

Développement back-end

Génération des PDF

Un membre peut télécharger un tirage depuis son historique. La route protégée récupère d'abord le résultat avec son identifiant et l'identifiant du propriétaire. Si aucun enregistrement ne correspond, l'API renvoie 404 et ne génère aucun document.

Dans le fichier `src/libraries/ResultPdfGenerator.php`

```
$options = new Options();
$options->set('defaultFont', 'DejaVu Sans');
$options->set('isRemoteEnabled', false);

$dmpdf = new Dmpdf($options);
$dmpdf->loadHtml($html, 'UTF-8');
$dmpdf->setPaper('A4', 'portrait');
$dmpdf->render();
$dmpdf->stream($filename, ['Attachment' => true]);
```

Construction du document

1. Le modèle charge le tirage et ses positions dans le bon ordre.
2. Le générateur échappe les valeurs avec `htmlspecialchars` avant de produire le HTML.
3. Une feuille de style crée l'en-tête, les informations du tirage et une carte par rune.
4. Dmpdf convertit le HTML en A4 et force le téléchargement du fichier.

Le chargement de ressources distantes est désactivé. Cela limite les dépendances externes et empêche le moteur PDF de récupérer une URL fournie dans les données. La police DejaVu Sans assure l'affichage des caractères accentués et des symboles disponibles.

Amélioration prévue : Je souhaite enrichir les PDF avec une couverture, une meilleure composition graphique, les interprétations liées à chaque position et une pagination plus travaillée pour les tirages longs

Développement back-end

Gestion des tirages et logique métier

Le tirage n'est pas un simple affichage aléatoire. Le serveur choisit des runes distinctes, attribue un rôle à chaque ordre, recherche l'interprétation adaptée puis enregistre le résultat uniquement lorsqu'un JWT facultatif permet d'identifier un membre.

Tirage	Nombre	Rôles utilisés
Nornes	3	passé, présent, futur
Cercle d'Odin	9	origine, passé, présent, obstacle, aide, conseil, futur, synthèse
Tirage du jour	3	passé, présent, futur ; renouvelé à 8 h 30

Dans le fichier principe appliqué dans DrawController

1. Valider la question et la catégorie.
2. Charger le format de tirage et la catégorie.
3. Sélectionner 3 ou 9 runes au hasard.
4. Associer un rôle à chaque ordre.
5. Chercher l'interprétation :
rune + catégorie + rôle + état.
6. Sauvegarder résultat et positions pour le membre.
7. Renvoyer le tirage complet en JSON.

La question est limitée à 255 caractères et la catégorie est obligatoire. Si le format, la catégorie ou le nombre de runes attendu manque, le serveur arrête le traitement avec un message contrôlé.

Le tirage du jour est différent : il est commun à tous. Avant 8 h 30, l'API utilise encore la date de la veille. Elle recherche d'abord un résultat daily existant et ne crée un nouveau tirage que s'il n'en existe pas pour cette date logique.

Cohérence : L'ordre est enregistré dans positions. Le front peut donc rejouer l'animation ou réafficher un ancien tirage sans perdre la place de chaque rune.

*Sécurité de l'application***Authentifier, protéger, valider et répondre proprement**

Point	Mise en œuvre dans le projet	Effet
Authentification	passwordhash / passwordverify, e-mail vérifié, JWT 1 h	Réduit l'usurpation de compte
Routes	middlewares member/admin et contrôle du propriétaire	Bloque les accès non autorisés
Environnement	.env ignoré par Git ; DB, JWT et Brevo séparés	Protège les secrets
Validation	e-mail valide, mot de passe ≥ 8 , tailles et valeurs autorisées	Refuse les données incohérentes
Erreurs	JSON uniforme, statuts HTTP, détail journalisé côté serveur	N'expose pas la trace technique

Mesures complémentaires

- Ratelimiter limite à cinq tentatives par minute et par adresse IP l'inscription, la connexion et les demandes sensibles.
- Les jetons de réinitialisation et de suppression sont produits avec random_bytes puis stockés sous forme hachée.
- CORS n'autorise que l'adresse FRONTEND_URL et seulement les méthodes et en-têtes nécessaires.
- Les en-têtes nosniff, DENY, Referrer-Policy et HSTS renforcent le navigateur ; HTTPS est obligatoire en production.
- La clé JWT doit être une chaîne hexadécimale d'au moins 64 caractères en production.

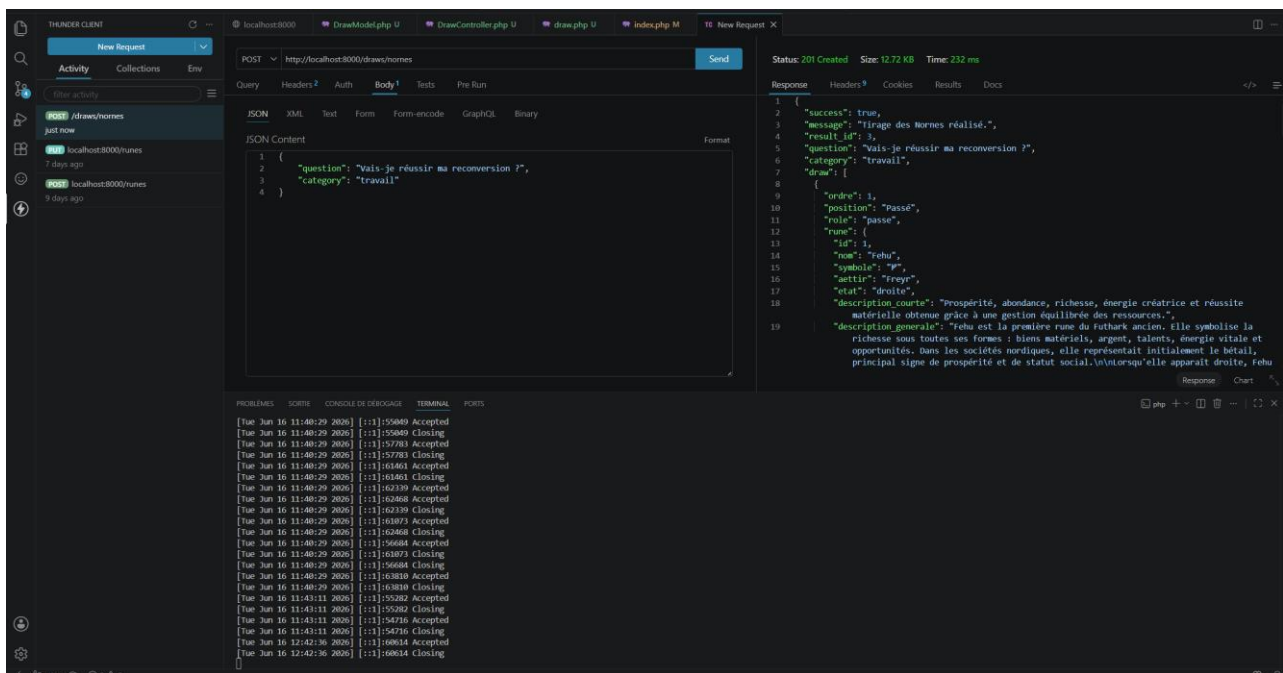
Le point d'entrée active le mode debug uniquement en environnement local. En cas d'exception imprévue, le détail est écrit dans le journal du serveur, tandis que le client reçoit seulement un message générique et un statut 500.

Réponse neutre : Pour le mot de passe oublié, l'API renvoie le même message que l'adresse existe ou non. Elle évite ainsi de révéler la liste des comptes inscrits.

Tests et validation

Thunder Client, jeux d'essais, vérifications

J'ai utilisé Thunder Client dans Visual Studio Code pour envoyer directement des requêtes à l'API. Cela me permet d'isoler le back-end du front, d'essayer différents corps JSON et de vérifier le statut ainsi que la réponse renvoyée.



POST /draws/normes : question et catégorie envoyées, tirage complet reçu avec le statut 201 Created.

Jeu d'essai	Résultat attendu
Connexion valide / mot de passe faux	200 avec JWT / 401 sans détail sensible
Route membre sans Bearer	401 Token manquant
Route admin avec rôle member	403 Accès admin refusé
Question de plus de 255 caractères	400 La question est trop longue
Résultat d'un autre membre	404, sans divulguer le propriétaire
Sixième tentative dans la minute	429 Trop de tentatives

Vérification fonctionnelle : Je contrôle à la fois le cas nominal et les refus : données invalides, jeton absent ou expiré, mauvais rôle et ressource introuvable.

Déploiement

De l'environnement local à la mise en ligne

Préparation de l'environnement

Le déploiement correspond au passage du projet depuis l'environnement local vers l'hébergement Hostinger. Dans ce projet, le site est composé de deux parties : le front-end, qui affiche l'interface utilisateur, et le back-end, qui gère les traitements, l'authentification, les tirages, les PDF et les échanges avec la base de données MySQL.

Le front-end et le back-end sont déployés séparément, mais ils doivent être configurés pour fonctionner ensemble. Le front-end est accessible depuis le domaine principal du site, par exemple <https://runes.esotheos.fr>, tandis que l'API back-end est appelée depuis un sous-domaine dédié, par exemple <https://api-runes.esotheos.fr>.

Avant la mise en ligne, le front-end doit être préparé pour la production. Les adresses locales utilisées pendant le développement sont remplacées par les vraies URLs de production. La variable d'environnement du front indique notamment l'adresse de l'API : **VITE_API_URL=https://api-runes.esotheos.fr**

Une fois cette configuration vérifiée, le front-end est compilé avec la commande : **npm run build**

Cette commande génère un dossier dist, qui contient les fichiers optimisés pour la production : HTML, CSS, JavaScript et ressources statiques. Ce dossier est ensuite envoyé sur Hostinger dans le répertoire lié au domaine public du site. Le serveur doit aussi être configuré pour que les routes React fonctionnent correctement. Ainsi, les pages comme /connexion, /tirage-gratuit, /glossaire-des-runes ou /dashboard-membre doivent toutes pouvoir charger l'application sans provoquer d'erreur 404 lors d'un rechargement de page.

Le back-end est également préparé pour la production. Le dépôt contient les fichiers composer.json, composer.lock et .env.exemple. Après l'envoi du projet sur Hostinger, les dépendances PHP sont installées avec : **composer install**

Le fichier **.env.exemple** sert de modèle pour créer le fichier **.env** réel.

Ce fichier contient les informations sensibles du projet : accès MySQL, clé JWT, URL du front-end, paramètres d'envoi d'e-mails et mode de production. Il ne doit pas être envoyé sur Git et ne doit pas être accessible publiquement.

La configuration générale sur Hostinger est la suivante :

Élément	Configuration
Front-end	Build de production généré avec npm run build
Domaine front	Domaine public du site, par exemple https://runes.esotheos.fr
Back-end	API PHP avec FlightPHP, Composer et variables d'environnement
Domaine API	Sous-domaine dédié, par exemple https://api-runes.esotheos.fr
Base de données	MySQL en utf8mb4, avec utilisateur dédié
Sécurité	.env non public, clé JWT forte, HTTPS actif
Communication	Le front appelle l'API grâce à l'URL définie dans ses variables d'environnement

Mise en ligne

La mise en ligne se fait donc en plusieurs étapes : créer la base de données de production, importer sa structure et les données de référence, déployer le back-end sur Hostinger, installer les dépendances Composer, renseigner le fichier .env, compiler le front-end, puis envoyer le dossier dist sur l'hébergement du domaine principal.

Après le déploiement, plusieurs tests sont nécessaires. Côté front-end, il faut vérifier l'affichage des pages, la navigation React, le responsive, les formulaires, les pages publiques et les espaces protégés. Côté back-end, il faut tester les routes de statut, l'inscription, la connexion, les tirages, l'historique membre, la génération PDF et les routes administrateur.

Un point important concerne la communication entre les deux parties. Le front-end doit appeler la bonne URL d'API, et le back-end doit autoriser le domaine du front dans sa configuration CORS. Sans cette correspondance, le site peut s'afficher correctement mais ne pas pouvoir se connecter à l'API.

En production, les erreurs détaillées ne doivent pas être visibles par les visiteurs. Elles sont consultées dans les journaux du serveur Hostinger, afin d'identifier les problèmes sans exposer d'informations sensibles. Des sauvegardes régulières de la base de données doivent également être prévues et conservées en dehors du répertoire public.

Le déploiement ne consiste donc pas seulement à mettre des fichiers en ligne. Il comprend la préparation du front-end, la configuration du back-end, la connexion entre les deux parties, la sécurisation des données, les tests fonctionnels et la surveillance après publication.

Évolutions et perspectives

Faire grandir Runes et Destin sans repartir de zéro

L'architecture séparée me permet d'ajouter de nouvelles interfaces et de nouveaux tirages tout en réutilisant la même API. Les évolutions sont classées selon leur valeur pour l'utilisateur et les dépendances techniques nécessaires.

Évolution	Principe	Travail back-end à prévoir
Marteau de Thor	Tirage à 5 runes	Nouveau format, rôles et interprétations
Tête de Mimir	Tirage à 7 runes	Ordres, positions et tests dédiés
Paielement	Nornes gratuit, autres tirages payants	Commandes, transactions, webhooks et factures
Talismans	Proposition selon le thème et les runes	Règles de correspondance et nouveaux contenus
Histoire / mythologie	Articles pédagogiques nordiques	Contenus administrables et catégories
PDF améliorés	Document plus immersif et complet	Nouveaux modèles, images, pagination

Le paiement sera ajouté seulement après avoir renforcé les tests et la journalisation. Le serveur ne devra jamais faire confiance au prix envoyé par le navigateur. Il recalculera le montant depuis le catalogue, vérifiera la signature du prestataire et enregistrera l'état de la commande à partir des notifications serveur.

Les nouveaux tirages pourront réutiliser results et positions. L'ajout portera surtout sur draws, les rôles attendus et les interprétations. Cette réutilisation confirme l'intérêt du modèle relationnel choisi.

Priorité : Je souhaite d'abord améliorer les PDF et ajouter les deux tirages déjà conçus, puis introduire le paiement lorsque le parcours complet pourra être testé de bout en bout.

Épilogue

Ce que ce projet m'a apporté

« Runes et Destin » m'a permis de réunir plusieurs parties de mon parcours : le graphisme, mon intérêt pour l'informatique, mon expérience humaine et une passion ancienne pour les runes. J'ai appris à passer d'une idée très large à une première version réellement utilisable.

Le front-end m'a appris à construire une interface responsive, accessible et cohérente. Le back-end m'a obligée à raisonner autrement : organiser les données, répartir les responsabilités, valider chaque entrée, protéger les routes et prévoir les erreurs. J'ai compris qu'une application ne se limite pas à ce que l'on voit à l'écran.

Ce projet m'a aussi appris à faire des choix. J'avais envie de développer immédiatement les paiements, les talismans, l'histoire et plusieurs tirages. J'ai choisi de terminer d'abord le cœur de l'application. Reporter une fonction n'est pas l'abandonner : c'est protéger la qualité du périmètre retenu.

Retour vers le développement après dix ans d'aide à domicile

Revenir vers un métier technique après dix ans d'aide à domicile représentait un vrai changement. Cette période m'a cependant apporté de la patience, de l'écoute et le sens des responsabilités. Je retrouve ces qualités dans ma manière de développer : comprendre le besoin, accompagner la personne et ne pas perdre de vue l'usage réel derrière le code.

Mes objectifs futurs

Je souhaite continuer à progresser comme développeuse web et web mobile, renforcer mes compétences en sécurité et en tests, et faire évoluer cette application après l'examen. À terme, j'aimerais proposer une plateforme complète autour des runes et de la culture nordique, puis réutiliser l'API pour une application mobile.

Ce dossier marque donc une étape importante : il présente les compétences acquises, mais aussi le retour vers un domaine qui me passionne et dans lequel je veux désormais construire la suite de mon parcours.

Conclusion personnelle : Après avoir longtemps gardé le développement comme une passion, j'ai aujourd'hui les bases, la méthode et surtout l'envie d'en faire durablement mon métier.